

MicroACT Code Explanation Report

README omitted by request. This report explains the remaining source/config files from core data flow to model internals, with code snippets before their explanations.

How To Read This Report

Each section starts with purpose and structure, then walks through the file in logical blocks. The snippets use real line numbers from the current repository. The explanations are intentionally lower-level than the earlier reading-order PDF: they describe what the lines do, what tensors/shapes mean, why a helper exists, and how each block connects to the rest of the system.

- Core runtime path: `config/config.py` -> `data/dataset.py` -> `train.py` -> `model/act_policy.py` -> `model/cvae.py` -> `model/backbone.py` and `model/transformer.py`.
- The visualization/export scripts are not part of training, but they reuse the same ACTCVAE model to inspect architecture and autograd behavior.
- Generated directories such as checkpoints, `onnx_exports`, `torchviz_exports`, dataset frame folders, and `__pycache__` are described conceptually instead of line-by-line.

requirements.txt

Purpose. Lists Python package dependencies needed to run the training, model, data loader, and optional Cellpose backbone.

Structure.

- Lines 1-5 are mandatory runtime dependencies.
- Lines 6-10 document and pin optional Cellpose support.

requirements.txt:1-5 - Core dependencies

```
1 | torch>=2.0
2 | torchvision>=0.15
3 | numpy>=1.24,<2
4 | pandas>=2.0
5 | pillow>=10.0
```

- torch and torchvision supply tensors, neural network layers, pretrained ResNet18, optimizers, and ONNX export.
- numpy and pandas handle numerical arrays and CSV logs.
- pillow loads microscope images from disk and converts/resizes them.

requirements.txt:6-10 - Optional Cellpose dependency

```
6 | # Optional - only needed for backbones that include "+cellpose".
7 | # Pin to 3.x for the lighter U-Net architecture (Cellpose 4 is a 304M-param
8 | # SAM-ViT). The numba dependency that breaks on import is bypassed: we
9 | # import only cellpose.resnet_torch.CPnet, not the full cellpose.models API.
10 | cellpose>=3.1,<4.0
```

- Cellpose is only required when the chosen backbone contains +cellpose.
- The version range intentionally stays below Cellpose 4 because this code expects the lighter Cellpose 3 CPnet architecture.
- The comment explains why the code imports a low-level Cellpose module instead of the higher-level package API.

.gitignore

Purpose. Defines which local files should not be committed. This is important in ML repos because generated artifacts and datasets can be huge.

Structure.

- Python caches and environments are ignored.
- ML artifacts and large data are ignored.
- Editor, OS, and local tooling files are ignored.

.gitignore:1-15 - Python build/cache and environment files

```
1 | # Python
2 | __pycache__/
3 | *.py[cod]
4 | *$py.class
5 | *.so
6 | .Python
7 |
8 | # Virtual environments
9 | .venv/
10 | venv/
11 | env/
12 | ENV/
13 |
14 | # Distribution / packaging
15 | build/
```

- The first group prevents Python bytecode, native extensions, and virtualenv folders from entering git.
- This keeps commits focused on source and avoids machine-specific environments.

.gitignore:17-36 - ML artifacts and dataset outputs

```
17 | *.egg-info/
18 | *.egg
19 |
20 | # Jupyter / IPython
21 | .ipynb_checkpoints/
22 |
23 | # Testing / coverage
24 | .pytest_cache/
```

```

25 | .mypy_cache/
26 | .ruff_cache/
27 | .coverage
28 | htmlcov/
29 |
30 | # ML artifacts (too large for git; rebuild from requirements + dataset)
31 | checkpoints/
32 | onnx_exports/
33 | torchviz_exports/
34 | *.pt
35 | *.pth
36 | *.ckpt

```

- Checkpoints, ONNX files, torchviz SVG output, pickle stats, and model weight extensions are ignored because they are generated and often large.
- Only saved_frames and saved_videos are ignored under dataset; CSV logs can still be tracked if desired.

.gitignore:38-57 - Editor, OS, and local tooling metadata

```

38 | *.onnx
39 | *.onnx.data
40 |
41 | # Dataset (use external storage / Git LFS, not git)
42 | dataset/saved_frames/
43 | dataset/saved_videos/
44 |
45 | # Editor / IDE
46 | .vscode/
47 | .idea/
48 | *.swp
49 | *.sw0
50 | *~
51 |
52 | # OS
53 | .DS_Store
54 | Thumbs.db
55 |
56 | # Tooling caches
57 | .codex

```

- These patterns avoid IDE folders, swap files, OS thumbnails, and local Codex metadata.
- The repository stays portable across machines and tools.

config/config.py

Purpose. Centralizes every shape, path, hyperparameter, and deployment placeholder so the rest of the code imports one consistent configuration module.

Structure.

- Path constants locate dataset, frames, checkpoints, and normalization stats.
- Robot/data constants define the vector dimensions and CSV columns.
- Model constants define the ACT/CVAE/Transformer dimensions.
- Training and rollout constants provide defaults used by train.py and deployment examples.

config/config.py:1-8 - Module docstring and Path import

```

1 | """Global configuration for MicroACT.
2 |
3 | Shapes here are tied to the current rig: 2 Sensapex uMp4 stages, 1 camera,
4 | no focusing motor and no pressure solenoids. To add those later, bump
5 | STATE_DIM / ACTION_DIM and extend CSV_STATE_COLS / CSV_ACTION_COLS.
6 | """
7 |
8 | from pathlib import Path

```

- The docstring states the physical rig assumption: two Sensapex stages and one camera.
- Path is used instead of raw strings so path joins are OS-aware and readable.

config/config.py:10-16 - Repository paths

```

10 | # ----- Paths -----
11 | REPO_ROOT = Path(__file__).resolve().parent.parent
12 | DATASET_ROOT = REPO_ROOT / "dataset"
13 | LOGS_DIR = DATASET_ROOT / "logs" # trial_N.csv
14 | FRAMES_DIR = DATASET_ROOT / "saved_frames" # trial_N/frame_NNNNNN.png
15 | CKPT_DIR = REPO_ROOT / "checkpoints"
16 | STATS_PATH = CKPT_DIR / "dataset_stats.pkl"

```

- REPO_ROOT is computed from this file location, so scripts can run from the repo without hardcoded absolute paths.

- DATASET_ROOT, LOGS_DIR, FRAMES_DIR, CKPT_DIR, and STATS_PATH are derived from REPO_ROOT and reused by dataset/training/checkpoint code.

config/config.py:18-40 - Robot vectors, images, and CSV schema

```

18 | # ----- Robot / data shapes -----
19 | # State = [x1, y1, z1, d1, x2, y2, z2, d2] (centered Sensapex counts)
20 | # Action = absolute target in the same 8-dim space.
21 | STATE_DIM = 8
22 | ACTION_DIM = 8
23 |
24 | NUM_CAMERAS = 1
25 | CAMERA_NAMES = ("cam_main",)
26 | IMAGE_HEIGHT = 240
27 | IMAGE_WIDTH = 320
28 |
29 | # CSV columns consumed from each trial_N.csv.
30 | # Anything else in the file (motor, image_path handled separately) is ignored.
31 | CSV_STATE_COLS = (
32 |     "current_x", "current_y", "current_z", "current_d",
33 |     "current_x2", "current_y2", "current_z2", "current_d2",
34 | )
35 | CSV_ACTION_COLS = (
36 |     "target_x", "target_y", "target_z", "target_d",
37 |     "target_x2", "target_y2", "target_z2", "target_d2",
38 | )
39 | CSV_IMAGE_COL = "image_path"
40 | CSV_TIMESTEP_COL = "timestep"

```

- STATE_DIM and ACTION_DIM are both 8: x/y/z/d for stage 1 plus x/y/z/d for stage 2.
- NUM_CAMERAS is currently 1; the model has some multi-camera reshaping support but config pins this rig to one camera.
- CSV_STATE_COLS and CSV_ACTION_COLS map named CSV columns into ordered numerical vectors. Changing modalities starts here.
- CSV_IMAGE_COL gives the optional path column; CSV_TIMESTEP_COL is declared for schema clarity even though dataset.py mostly uses row index and fallback paths.

config/config.py:42-54 - ACT model hyperparameters

```

42 | # ----- ACT model hyperparameters -----
43 | CHUNK_SIZE = 100 # k: number of future actions predicted per inference
44 | HIDDEN_DIM = 512
45 | DIM_FEEDFORWARD = 3200
46 | ENC_LAYERS = 4
47 | DEC_LAYERS = 7
48 | NHEAD = 8
49 | DROPOUT = 0.1
50 | LATENT_DIM = 32 # CVAE style-latent dimension
51 | KL_WEIGHT = 10.0 # beta on the KL term (ACT paper default)
52 |
53 | BACKBONE = "resnet18"
54 | BACKBONE_PRETRAINED = True

```

- CHUNK_SIZE is the number of future actions predicted from one observation.
- HIDDEN_DIM, DIM_FEEDFORWARD, ENC_LAYERS, DEC_LAYERS, NHEAD, and DROPOUT configure transformer capacity.
- LATENT_DIM controls the CVAE style code size; KL_WEIGHT controls how strongly that latent is regularized toward a standard normal prior.
- BACKBONE and BACKBONE_PRETRAINED set the default image encoder choice.

config/config.py:56-69 - Training and rollout defaults

```

56 | # ----- Training -----
57 | BATCH_SIZE = 8
58 | NUM_EPOCHS = 2000
59 | LR = 1e-5
60 | LR_BACKBONE = 1e-5
61 | WEIGHT_DECAY = 1e-4
62 | SEED = 0
63 | DEVICE = "cuda"
64 |
65 | # ----- Rollout -----
66 | OPEN_LOOP_HORIZON = 8 # actions consumed per inference during rollout
67 | CONTROL_HZ = 5.0 # matches logger_node default log_interval_ms = 200
68 | TEMPORAL_AGG = True # ACT-style exponential ensembling across chunks
69 | TEMPORAL_AGG_K = 0.01

```

- Training constants are argparse defaults in train.py.
- DEVICE defaults to cuda, but train.py falls back to CPU if CUDA is unavailable.
- Rollout constants describe intended deployment behavior; temporal aggregation flags are placeholders for ROS2-side rollout logic.

data/dataset.py

Purpose. Turns recorded trials into PyTorch samples: normalized image, normalized current state, normalized future action chunk, and a padding mask.

Structure.

- TrialData holds one loaded trial.
- Loading helpers discover and validate CSV logs.
- Image helpers resolve stored paths and fall back safely.
- compute_norm_stats creates dataset-wide normalization.
- EpisodicDataset emits one sample per trial timestep.
- build_dataset wires discovery, stats caching, and dataset construction together.

data/dataset.py:1-28 - Docstring, imports, and config import

```
1 | """CSV + image dataset loader for MicroACT.
2 |
3 | Each trial_N.csv under DATASET_ROOT/logs has one row per timestep with the
4 | columns declared in config.config. Images live at
5 | DATASET_ROOT/saved_frames/trial_N/frame_NNNNNN.png (path is also stored in
6 | the `image_path` CSV column).
7 |
8 | Each sample emitted here is:
9 |     image : (num_cameras, 3, H, W) float32, ImageNet-normalized
10 |     qpos  : (STATE_DIM,) float32, dataset-normalized
11 |     action : (CHUNK_SIZE, ACTION_DIM) float32, dataset-normalized, zero-padded
12 |     is_pad : (CHUNK_SIZE,) bool, True where action was padded
13 | """
14 |
15 | from __future__ import annotations
16 |
17 | import pickle
18 | import warnings
19 | from pathlib import Path
20 | from typing import List, NamedTuple, Optional
21 |
22 | import numpy as np
23 | import pandas as pd
24 | import torch
25 | from PIL import Image
26 | from torch.utils.data import Dataset
27 |
28 | from config import config as C
```

- The docstring defines the contract of one dataset item, including exact tensor shapes.
- pickle is used for cached normalization stats; warnings is used to warn once about missing images.
- numpy/pandas/PIL/torch form the file-to-tensor data stack.
- The config module is imported as C so all shapes and paths come from one place.

data/dataset.py:31-36 - TrialData container

```
31 | class TrialData(NamedTuple):
32 |     trial_id: int
33 |     states: np.ndarray          # (T, STATE_DIM)
34 |     actions: np.ndarray        # (T, ACTION_DIM)
35 |     image_paths: List[str]     # length T; '' means no path recorded
36 |     length: int
```

- NamedTuple makes each loaded trial immutable-ish and self-documenting.
- states and actions are dense arrays shaped by config.STATE_DIM and config.ACTION_DIM.
- image_paths preserves the raw CSV path strings, including empty strings when absent.
- length avoids repeatedly asking arrays or dataframes for their first dimension.

data/dataset.py:43-50 - discover_trials

```
43 | def discover_trials(logs_dir: Path = C.LOGS_DIR) -> List[Path]:
44 |     files = sorted(
45 |         logs_dir.glob("trial_*.csv"),
46 |         key=lambda p: int(p.stem.split("_")[-1]),
47 |     )
48 |     if not files:
49 |         raise FileNotFoundError(f"No trial_*.csv found under {logs_dir}")
50 |     return files
```

- glob finds files named trial_*.csv under the logs directory.

- The sort key extracts the numeric suffix so trial_10 sorts after trial_9, not after trial_1.
- A missing dataset is treated as a hard error because training cannot proceed without trials.

data/dataset.py:53-68 - load_trial

```

53 | def load_trial(csv_path: Path) -> TrialData:
54 |     df = pd.read_csv(csv_path)
55 |     trial_id = int(csv_path.stem.split("_")[-1])
56 |
57 |     missing = [c for c in (*C.CSV_STATE_COLS, *C.CSV_ACTION_COLS) if c not in df.columns]
58 |     if missing:
59 |         raise ValueError(f"{csv_path.name} missing columns: {missing}")
60 |
61 |     states = df[list(C.CSV_STATE_COLS)].to_numpy(dtype=np.float32)
62 |     actions = df[list(C.CSV_ACTION_COLS)].to_numpy(dtype=np.float32)
63 |     raw_paths = (
64 |         df[C.CSV_IMAGE_COL].fillna("").astype(str).tolist()
65 |         if C.CSV_IMAGE_COL in df.columns
66 |         else [""] * len(df)
67 |     )
68 |     return TrialData(trial_id, states, actions, raw_paths, length=len(df))

```

- pandas reads the CSV into a dataframe and the trial id is parsed from the filename.
- The missing-column check guarantees all configured state/action columns exist before arrays are created.
- State and action columns are converted to float32 arrays, matching PyTorch training expectations.
- image_path is optional; when absent, an empty path list triggers fallback path construction later.

data/dataset.py:75-98 - Image path resolution and loading

```

75 | def _resolve_image_path(raw: str, trial_id: int, t: int) -> Optional[Path]:
76 |     """Try a few conventions to find the frame file. Return None if unresolved."""
77 |     # Conventional fallback: saved_frames/trial_N/frame_NNNNNN.png
78 |     fallback = C.FRAMES_DIR / f"trial_{trial_id}" / f"frame_{t:06d}.png"
79 |
80 |     raw = (raw or "").strip()
81 |     if not raw:
82 |         return fallback if fallback.exists() else None
83 |
84 |     p = Path(raw)
85 |     if p.is_absolute():
86 |         return p if p.exists() else None
87 |     for base in (C.REPO_ROOT, C.DATASET_ROOT, C.FRAMES_DIR):
88 |         q = base / p
89 |         if q.exists():
90 |             return q
91 |     return fallback if fallback.exists() else None
92 |
93 |
94 | def _load_image(path: Optional[Path], h: int, w: int) -> np.ndarray:
95 |     if path is None:
96 |         return np.zeros((h, w, 3), dtype=np.uint8)
97 |     img = Image.open(path).convert("RGB").resize((w, h), Image.BILINEAR)
98 |     return np.asarray(img, dtype=np.uint8)

```

- _resolve_image_path tries the conventional saved_frames path first when the CSV cell is empty.
- Absolute CSV paths are used only if they exist; relative paths are tried against repo root, dataset root, and frames root.
- If nothing resolves, None is returned. _load_image then turns None into a black zero image instead of crashing.
- Resolved images are converted to RGB and resized to the configured width and height.

data/dataset.py:105-120 - Dataset normalization statistics

```

105 | def compute_norm_stats(trials: List[TrialData]) -> dict:
106 |     all_states = np.concatenate([t.states for t in trials], axis=0)
107 |     all_actions = np.concatenate([t.actions for t in trials], axis=0)
108 |
109 |     # Clip std so constant dims don't divide by zero.
110 |     state_std = np.clip(all_states.std(0), 1e-2, None)
111 |     action_std = np.clip(all_actions.std(0), 1e-2, None)
112 |
113 |     return {
114 |         "qpos_mean": all_states.mean(0).astype(np.float32),
115 |         "qpos_std": state_std.astype(np.float32),
116 |         "action_mean": all_actions.mean(0).astype(np.float32),
117 |         "action_std": action_std.astype(np.float32),
118 |         "image_mean": np.array([0.485, 0.456, 0.406], dtype=np.float32),
119 |         "image_std": np.array([0.229, 0.224, 0.225], dtype=np.float32),
120 |     }

```

- All trial states/actions are concatenated along time so mean and std describe the full dataset.
- np.clip prevents division by zero for constant dimensions by forcing std to be at least 1e-2.
- Image mean/std are ImageNet constants because the supported image backbones are ImageNet/DINO-style encoders.

data/dataset.py:127-146 - EpisodicDataset initialization

```

127 | class EpisodicDataset(Dataset):
128 |     """One sample per (trial, start_t). Actions past the trial end are zero-padded."""
129 |
130 |     def __init__(
131 |         self,
132 |         trials: List[TrialData],
133 |         norm_stats: dict,
134 |         chunk_size: int = C.CHUNK_SIZE,
135 |         image_hw: tuple = (C.IMAGE_HEIGHT, C.IMAGE_WIDTH),
136 |     ):
137 |         self.trials = trials
138 |         self.norm_stats = norm_stats
139 |         self.chunk_size = chunk_size
140 |         self.image_h, self.image_w = image_hw
141 |         self.index = [
142 |             (ti, t)
143 |             for ti, tr in enumerate(trials)
144 |             for t in range(tr.length)
145 |         ]
146 |         self._warned_missing_image = False

```

- The dataset stores loaded trials and normalization stats rather than repeatedly reading files.
- `self.index` flattens all (trial, timestep) pairs into one list, making each timestep an independent training sample.
- `_warned_missing_image` suppresses repeated warnings so a bad path pattern does not spam logs.

data/dataset.py:148-193 - EpisodicDataset `__len__` and `__getitem__`

```

148 |     def __len__(self) -> int:
149 |         return len(self.index)
150 |
151 |     def __getitem__(self, i: int) -> dict:
152 |         trial_idx, t = self.index[i]
153 |         trial = self.trials[trial_idx]
154 |
155 |         # ---- State at t ----
156 |         qpos = trial.states[t]
157 |
158 |         # ---- Future action chunk, zero-padded ----
159 |         end = min(t + self.chunk_size, trial.length)
160 |         avail = end - t
161 |         action = np.zeros((self.chunk_size, C.ACTION_DIM), dtype=np.float32)
162 |         action[:avail] = trial.actions[t:end]
163 |         is_pad = np.zeros(self.chunk_size, dtype=bool)
164 |         is_pad[avail:] = True
165 |
166 |         # ---- Image at t ----
167 |         raw = trial.image_paths[t] if t < len(trial.image_paths) else ""
168 |         path = _resolve_image_path(raw, trial.trial_id, t)
169 |         if path is None and not self._warned_missing_image:
170 |             warnings.warn(
171 |                 f"Trial {trial.trial_id}: image at t={t} unresolved "
172 |                 f"(csv='{raw}') - returning zeros. Further warnings suppressed.",
173 |                 stacklevel=2,
174 |             )
175 |             self._warned_missing_image = True
176 |         img = _load_image(path, self.image_h, self.image_w)
177 |
178 |         # ---- Normalize ----
179 |         img = img.astype(np.float32) / 255.0
180 |         img = (img - self.norm_stats["image_mean"]) / self.norm_stats["image_std"]
181 |         img = np.transpose(img, (2, 0, 1)) # HWC -> CHW
182 |         img = img[None] # (num_cam=1, C, H, W)
183 |
184 |         qpos_n = (qpos - self.norm_stats["qpos_mean"]) / self.norm_stats["qpos_std"]
185 |         action_n = (action - self.norm_stats["action_mean"]) / self.norm_stats["action_std"]
186 |         action_n[is_pad] = 0.0 # keep padded positions clean
187 |
188 |         return {
189 |             "image": torch.from_numpy(img).float(),
190 |             "qpos": torch.from_numpy(qpos_n).float(),
191 |             "action": torch.from_numpy(action_n).float(),
192 |             "is_pad": torch.from_numpy(is_pad),
193 |         }

```

- `__len__` reports the number of flattened timestep samples.
- `__getitem__` recovers the trial and timestep, takes `qpos` at `t`, and builds an action chunk from `t` through `t + chunk_size`.
- Actions beyond the end of a trial remain zeros and `is_pad` marks those positions True.
- Image data is normalized from uint8 RGB into float32 ImageNet-normalized CHW format, then a `num_camera` dimension is added.
- `qpos` and `actions` are normalized with dataset stats; padded action rows are reset to zero so padding does not contain normalized mean offsets.
- The return dict matches what `train.py` expects: `image`, `qpos`, `action`, `is_pad`.

data/dataset.py:200-217 - build_dataset convenience function

```
200 | def build_dataset(  
201 |     logs_dir: Path = C.LOGS_DIR,  
202 |     stats_path: Path = C.STATS_PATH,  
203 |     recompute_stats: bool = False,  
204 | ) -> EpisodicDataset:  
205 |     csv_paths = discover_trials(logs_dir)  
206 |     trials = [load_trial(p) for p in csv_paths]  
207 |  
208 |     if stats_path.exists() and not recompute_stats:  
209 |         with open(stats_path, "rb") as f:  
210 |             stats = pickle.load(f)  
211 |     else:  
212 |         stats = compute_norm_stats(trials)  
213 |         stats_path.parent.mkdir(parents=True, exist_ok=True)  
214 |         with open(stats_path, "wb") as f:  
215 |             pickle.dump(stats, f)  
216 |  
217 |     return EpisodicDataset(trials, stats)
```

- This function is the public entry point used by train.py.
- It discovers CSVs, loads each trial, loads or recomputes stats, stores stats if needed, and returns an EpisodicDataset.
- train.py passes recompute_stats=True so each run reflects the current dataset.

utils.py

Purpose. Contains small, reusable training utilities that keep train.py focused on orchestration.

Structure.

- Seeding makes splits and initialization more reproducible.
- Optimizer grouping lets the backbone use a different learning rate.
- Checkpoint helpers serialize model/optimizer/epoch state.
- AverageMeter and format_meters aggregate scalar losses.

utils.py:1-18 - Imports and set_seed

```
1 | """Small training helpers: seeding, optimizer builder, checkpoint IO, meters."""  
2 |  
3 | from __future__ import annotations  
4 |  
5 | import random  
6 | from pathlib import Path  
7 | from typing import Optional  
8 |  
9 | import numpy as np  
10 | import torch  
11 |  
12 |  
13 | def set_seed(seed: int) -> None:  
14 |     random.seed(seed)  
15 |     np.random.seed(seed)  
16 |     torch.manual_seed(seed)  
17 |     if torch.cuda.is_available():  
18 |         torch.cuda.manual_seed_all(seed)
```

- random, numpy, and torch all have separate random number generators, so each one is seeded.
- If CUDA exists, all CUDA devices are seeded too.
- This improves reproducibility but does not guarantee bit-exact determinism for every GPU operation.

utils.py:21-36 - build_optimizer

```
21 | def build_optimizer(  
22 |     policy: torch.nn.Module,  
23 |     lr: float,  
24 |     lr_backbone: float,  
25 |     weight_decay: float,  
26 | ) -> torch.optim.Optimizer:  
27 |     """AdamW with two param groups: backbone (lower LR) vs. everything else."""  
28 |     backbone_params = list(policy.model.backbone.parameters())  
29 |     backbone_ids = {id(p) for p in backbone_params}  
30 |     other_params = [p for p in policy.parameters() if id(p) not in backbone_ids]  
31 |  
32 |     param_groups = [  
33 |         {"params": [p for p in backbone_params if p.requires_grad], "lr": lr_backbone},  
34 |         {"params": [p for p in other_params if p.requires_grad], "lr": lr},  
35 |     ]  
36 |     return torch.optim.AdamW(param_groups, lr=lr, weight_decay=weight_decay)
```

- Backbone parameters are collected from `policy.model.backbone`.
- `id(p)` is used to identify those exact `Parameter` objects, so the rest of the model can be separated into `other_params`.
- Only parameters with `requires_grad=True` are included, which matters for frozen DINOv2/Cellpose backbones.
- AdamW receives two parameter groups: one for backbone LR and one for everything else.

`utils.py:39-63` - `save_checkpoint` and `load_checkpoint`

```

39 | def save_checkpoint(
40 |     path: Path,
41 |     policy: torch.nn.Module,
42 |     optimizer: Optional[torch.optim.Optimizer] = None,
43 |     epoch: Optional[int] = None,
44 | ) -> None:
45 |     path.parent.mkdir(parents=True, exist_ok=True)
46 |     ckpt = {"policy": policy.state_dict(), "epoch": epoch}
47 |     if optimizer is not None:
48 |         ckpt["optimizer"] = optimizer.state_dict()
49 |     torch.save(ckpt, path)
50 |
51 |
52 | def load_checkpoint(
53 |     path: Path,
54 |     policy: torch.nn.Module,
55 |     optimizer: Optional[torch.optim.Optimizer] = None,
56 |     map_location: Optional[str] = None,
57 | ) -> int:
58 |     """Loads weights (and optimizer state if provided). Returns the saved epoch."""
59 |     ckpt = torch.load(path, map_location=map_location)
60 |     policy.load_state_dict(ckpt["policy"])
61 |     if optimizer is not None and "optimizer" in ckpt:
62 |         optimizer.load_state_dict(ckpt["optimizer"])
63 |     return ckpt.get("epoch") or 0

```

- `save_checkpoint` creates the parent directory, then saves `policy.state_dict` and `epoch`.
- Optimizer state is optional, so the same helper can save training checkpoints or lighter model-only checkpoints.
- `load_checkpoint` restores weights and optionally optimizer state, then returns the stored epoch so training can resume at the right point.

`utils.py:66-87` - `AverageMeter` and `format_meters`

```

66 | class AverageMeter:
67 |     """Running mean for scalar metrics."""
68 |
69 |     def __init__(self):
70 |         self.sum = 0.0
71 |         self.count = 0
72 |
73 |     def update(self, val: float, n: int = 1) -> None:
74 |         self.sum += float(val) * n
75 |         self.count += n
76 |
77 |     @property
78 |     def avg(self) -> float:
79 |         return self.sum / self.count if self.count else 0.0
80 |
81 |     def reset(self) -> None:
82 |         self.sum = 0.0
83 |         self.count = 0
84 |
85 |
86 | def format_meters(meters: dict) -> str:
87 |     return " ".join(f"{k}={m.avg:.4f}" for k, m in meters.items())

```

- `AverageMeter` tracks weighted sums and counts so batch losses can be averaged by sample count.
- `avg` safely returns 0 if nothing has been added.
- `format_meters` turns a dictionary of meters into compact training log text.

train.py

Purpose. The command-line training entry point. It wires together config defaults, dataset loading, splitting, dataloaders, policy creation, optimizer creation, epoch loops, validation, and checkpointing.

Structure.

- `parse_args` defines the CLI and defaults.
- `run_epoch` handles one train or validation pass.
- `main` performs setup, splitting, loader construction, model/optimizer construction, resume, and checkpoint loop.

train.py:1-29 - Docstring and imports

```

1 | """Train MicroACT on the Sensapex dataset.
2 |
3 | Examples:
4 |     python train.py                                # defaults from config.config
5 |     python train.py --epochs 200 --batch-size 16
6 |     python train.py --resume checkpoints/policy_last.pt
7 |     python train.py --no-pretrained                # skip ImageNet weight download
8 | """
9 |
10 | from __future__ import annotations
11 |
12 | import argparse
13 | from collections import defaultdict
14 | from pathlib import Path
15 |
16 | import torch
17 | from torch.utils.data import DataLoader, Subset, random_split
18 |
19 | from config import config as C
20 | from data.dataset import build_dataset
21 | from model.act_policy import build_policy
22 | from utils import (
23 |     AverageMeter,
24 |     build_optimizer,
25 |     format_meters,
26 |     load_checkpoint,
27 |     save_checkpoint,
28 |     set_seed,
29 | )

```

- The docstring shows common invocations.
- argparse handles CLI flags; defaultdict stores metric meters.
- DataLoader, Subset, and random_split create training and validation iterators.
- Local imports connect this script to config, dataset construction, policy construction, and utilities.

train.py:32-58 - parse_args CLI definition

```

32 | def parse_args() -> argparse.Namespace:
33 |     p = argparse.ArgumentParser()
34 |     p.add_argument("--epochs", type=int, default=C.NUM_EPOCHS)
35 |     p.add_argument("--batch-size", type=int, default=C.BATCH_SIZE)
36 |     p.add_argument("--lr", type=float, default=C.LR)
37 |     p.add_argument("--lr-backbone", type=float, default=C.LR_BACKBONE)
38 |     p.add_argument("--weight-decay", type=float, default=C.WEIGHT_DECAY)
39 |     p.add_argument("--seed", type=int, default=C.SEED)
40 |     p.add_argument("--device", type=str, default=C.DEVICE)
41 |     p.add_argument("--val-split", type=float, default=0.1)
42 |     p.add_argument("--val-by-trial", action="store_true",
43 |                    help="Split val set by trial instead of by timestep. "
44 |                    "Avoids adjacent-timestep leakage but needs >=10 trials "
45 |                    "for a stable val signal. See README.md §9.9.")
46 |     p.add_argument("--num-workers", type=int, default=4)
47 |     p.add_argument("--save-every", type=int, default=100,
48 |                    help="Save a numbered checkpoint every N epochs (in addition to last/best).")
49 |     p.add_argument("--ckpt-dir", type=Path, default=C.CKPT_DIR)
50 |     p.add_argument("--resume", type=Path, default=None)
51 |     p.add_argument("--no-pretrained", action="store_true",
52 |                    help="Disable ImageNet pretrained weights on the backbone.")
53 |     p.add_argument("--backbone", type=str, default=C.BACKBONE,
54 |                    help="Image backbone: resnet18 | dinov2_vits14 | dinov2_vitb14 | dinov2_vitl14. "
55 |                    "DINOv2 variants download from torch.hub on first use.")
56 |     p.add_argument("--unfreeze-backbone", action="store_true",
57 |                    help="DINOv2 backbones are frozen by default. Pass this to fine-tune them.")
58 |     return p.parse_args()

```

- Most defaults come directly from config/config.py, so config remains the source of truth.
- --val-by-trial switches from random timestep splitting to holding out whole trials.
- --resume points to a checkpoint; --no-pretrained avoids ResNet weight download; --backbone selects image encoder.
- --unfreeze-backbone lets frozen pretrained encoders become trainable when desired.

train.py:61-84 - run_epoch

```

61 | def run_epoch(policy, loader, optimizer, device, train: bool) -> dict:
62 |     policy.train(train)
63 |     meters: dict = defaultdict(AverageMeter)
64 |
65 |     for batch in loader:
66 |         image = batch["image"].to(device, non_blocking=True)
67 |         qpos = batch["qpos"].to(device, non_blocking=True)
68 |         action = batch["action"].to(device, non_blocking=True)
69 |         is_pad = batch["is_pad"].to(device, non_blocking=True)
70 |
71 |         if train:
72 |             loss_dict = policy(image, qpos, action, is_pad)
73 |             optimizer.zero_grad()

```

```

74 |         loss_dict["loss"].backward()
75 |         optimizer.step()
76 |     else:
77 |         with torch.no_grad():
78 |             loss_dict = policy(image, qpos, action, is_pad)
79 |
80 |         bs = qpos.size(0)
81 |         for k, v in loss_dict.items():
82 |             meters[k].update(v.item(), bs)
83 |
84 |     return meters

```

- `policy.train(train)` toggles train/eval mode for dropout, normalization, and custom frozen-backbone behavior.
- Each batch tensor is moved to the selected device with `non_blocking=True` to benefit from pinned memory when available.
- During training, the policy returns a loss dict; `loss.backward` computes gradients and `optimizer.step` updates parameters.
- During validation, `torch.no_grad` disables gradient tracking to save memory and compute.
- Meters aggregate loss, l1, and kl across the epoch using batch size as weight.

train.py:87-98 - main setup and dataset build

```

87 | def main() -> None:
88 |     args = parse_args()
89 |     set_seed(args.seed)
90 |
91 |     device = args.device
92 |     if device.startswith("cuda") and not torch.cuda.is_available():
93 |         print(f"[warn] CUDA unavailable, falling back to cpu")
94 |         device = "cpu"
95 |
96 |     # ---- Data ----
97 |     full_ds = build_dataset(recompute_stats=True)
98 |

```

- Arguments are parsed and the random seed is set before splitting.
- CUDA fallback prevents a crash when the default device is cuda but no GPU is visible.
- `build_dataset(recompute_stats=True)` rebuilds normalization stats from the current logs each run.

train.py:99-124 - Validation split logic

```

99 |     if args.val_by_trial:
100 |         n_trials = len(full_ds.trials)
101 |         n_val_trials = max(1, int(round(n_trials * args.val_split)))
102 |         if n_trials - n_val_trials < 1:
103 |             raise SystemExit(
104 |                 f"--val-by-trial needs at least 2 trials; got {n_trials}. "
105 |                 f"Drop the flag to fall back to timestep-level random_split."
106 |             )
107 |         perm = torch.randperm(
108 |             n_trials, generator=torch.Generator().manual_seed(args.seed)
109 |         ).tolist()
110 |         val_trial_set = set(perm[:n_val_trials])
111 |         train_idx = [i for i, (ti, _) in enumerate(full_ds.index) if ti not in val_trial_set]
112 |         val_idx = [i for i, (ti, _) in enumerate(full_ds.index) if ti in val_trial_set]
113 |         train_ds, val_ds = Subset(full_ds, train_idx), Subset(full_ds, val_idx)
114 |         val_trial_ids = sorted(full_ds.trials[ti].trial_id for ti in val_trial_set)
115 |         print(f"trial-level split: held-out trials = {val_trial_ids}")
116 |     else:
117 |         val_n = max(1, int(round(len(full_ds) * args.val_split)))
118 |         train_n = len(full_ds) - val_n
119 |         train_ds, val_ds = random_split(
120 |             full_ds, [train_n, val_n],
121 |             generator=torch.Generator().manual_seed(args.seed),
122 |         )
123 |
124 |     print(f"dataset: train={len(train_ds)} val={len(val_ds)} (total {len(full_ds)})")

```

- Trial-level splitting holds out entire trials to reduce adjacent-timestep leakage.
- The code verifies at least one training trial remains.
- It creates index lists by checking the trial index stored in `full_ds.index`.
- The default branch randomly splits individual timestep samples, which is easier but can be optimistic.

train.py:126-144 - DataLoaders, policy, and optimizer

```

126 |     loader_kwargs = dict(
127 |         batch_size=args.batch_size,
128 |         num_workers=args.num_workers,
129 |         pin_memory=(device.startswith("cuda")),
130 |         persistent_workers=args.num_workers > 0,
131 |     )
132 |     train_loader = DataLoader(train_ds, shuffle=True, **loader_kwargs)
133 |     val_loader = DataLoader(val_ds, shuffle=False, **loader_kwargs)
134 |

```

```

135 | # ---- Policy + optimizer ----
136 | policy = build_policy(
137 |     stats=full_ds.norm_stats,
138 |     pretrained_backbone=not args.no_pretrained,
139 |     backbone_name=args.backbone,
140 |     freeze_backbone=not args.unfreeze_backbone,
141 | ).to(device)
142 | print(f"backbone: {args.backbone} "
143 |       f"(frozen={not args.unfreeze_backbone if args.backbone != 'resnet18' else False})")
144 | optimizer = build_optimizer(policy, args.lr, args.lr_backbone, args.weight_decay)

```

- loader_kwargs centralizes batch size, worker count, pinned memory, and persistent worker behavior.
- Training loader shuffles; validation loader does not.
- build_policy receives dataset stats so normalization buffers are embedded inside the model.
- Backbone options are passed through to the CVAE; build_optimizer applies separate LR groups.

train.py:146-176 - Resume and checkpointed training loop

```

146 | start_epoch = 0
147 | best_val = float("inf")
148 | if args.resume is not None and args.resume.exists():
149 |     start_epoch = load_checkpoint(args.resume, policy, optimizer, map_location=device)
150 |     print(f"resumed from {args.resume} at epoch {start_epoch}")
151 |
152 | # ---- Loop ----
153 | ckpt_last = args.ckpt_dir / "policy_last.pt"
154 | ckpt_best = args.ckpt_dir / "policy_best.pt"
155 |
156 | for epoch in range(start_epoch, args.epochs):
157 |     tr = run_epoch(policy, train_loader, optimizer, device, train=True)
158 |     vl = run_epoch(policy, val_loader, optimizer, device, train=False)
159 |
160 |     print(
161 |         f"[epoch {epoch+1:4d}]/{args.epochs}] "
162 |         f"train {format_meters(tr)} | val {format_meters(vl)}"
163 |     )
164 |
165 |     save_checkpoint(ckpt_last, policy, optimizer, epoch + 1)
166 |     if vl["loss"].avg < best_val:
167 |         best_val = vl["loss"].avg
168 |         save_checkpoint(ckpt_best, policy, optimizer, epoch + 1)
169 |
170 |     if (epoch + 1) % args.save_every == 0:
171 |         save_checkpoint(
172 |             args.ckpt_dir / f"policy_epoch{epoch+1}.pt",
173 |             policy, optimizer, epoch + 1,
174 |         )
175 |
176 |     print(f"done. best val loss: {best_val:.4f}")

```

- start_epoch and best_val support resuming and best-checkpoint tracking.
- Each epoch runs training then validation and prints formatted metrics.
- policy_last.pt is written every epoch; policy_best.pt updates when validation loss improves.
- Numbered checkpoints are written at the configured save interval.

train.py:179-180 - Script entry point

```

179 | if __name__ == "__main__":
180 |     main()

```

- The if __name__ == "__main__" guard means main() runs only when this file is executed as a script, not when imported.

model/act_policy.py

Purpose. Wraps the raw ACTCVAE with training loss, normalization buffers, checkpoint-friendly stats, and a single-sample numpy inference API for rollout.

Structure.

- ACTPolicy owns ACTCVAE and stats buffers.
- forward either computes training losses or returns predictions.
- _compute_loss implements masked L1 plus KL.
- inference preprocesses raw image/qpos and denormalizes predicted actions.
- build_policy loads stats if they are not provided.

model/act_policy.py:1-23 - Docstring and imports

```

1 | """ACT policy wrapper: loss computation (training) + inference helper (rollout).
2 |
3 | Adds on top of ACTCVAE:
4 | - Dataset normalization stats registered as buffers, so a saved checkpoint
5 |   is self-contained and does not need a separate stats file at load time.
6 | - Training-time loss: masked L1 on the action chunk + KL(N(mu, sigma^2) || N(0, I)).
7 | - Numpy-in / numpy-out `.inference()` for the ROS2 rollout loop.
8 | """
9 |
10 | from __future__ import annotations
11 |
12 | import pickle
13 | from pathlib import Path
14 | from typing import Optional
15 |
16 | import numpy as np
17 | import torch
18 | import torch.nn as nn
19 | import torch.nn.functional as F
20 | from PIL import Image
21 |
22 | from config import config as C
23 | from model.cvae import ACTCVAE

```

- The docstring names the three responsibilities this wrapper adds on top of ACTCVAE.
- pickle and Path support loading cached stats.
- numpy/PIL handle raw inference input; torch/nn/F handle model, buffers, and loss math.

model/act_policy.py:26-46 - ACTPolicy initialization and buffers

```

26 | class ACTPolicy(nn.Module):
27 |     def __init__(
28 |         self,
29 |         stats: dict,
30 |         kl_weight: float = C.KL_WEIGHT,
31 |         **cvae_kwargs,
32 |     ):
33 |         super().__init__()
34 |         self.model = ACTCVAE(**cvae_kwargs)
35 |         self.kl_weight = kl_weight
36 |
37 |         self.register_buffer("qpos_mean", torch.from_numpy(stats["qpos_mean"]))
38 |         self.register_buffer("qpos_std", torch.from_numpy(stats["qpos_std"]))
39 |         self.register_buffer("action_mean", torch.from_numpy(stats["action_mean"]))
40 |         self.register_buffer("action_std", torch.from_numpy(stats["action_std"]))
41 |         self.register_buffer(
42 |             "image_mean", torch.from_numpy(stats["image_mean"]).view(3, 1, 1)
43 |         )
44 |         self.register_buffer(
45 |             "image_std", torch.from_numpy(stats["image_std"]).view(3, 1, 1)
46 |         )

```

- self.model is the actual ACTCVAE network.
- kl_weight controls how much the KL regularizer contributes to total loss.
- register_buffer stores normalization tensors in the module state without treating them as trainable parameters.
- Because buffers live in the state_dict and move with .to(device), checkpoints and inference are cleaner.

model/act_policy.py:52-65 - forward dispatch

```

52 |     def forward(
53 |         self,
54 |         image: torch.Tensor,                # pre-normalized by the DataLoader
55 |         qpos: torch.Tensor,
56 |         actions: Optional[torch.Tensor] = None,
57 |         is_pad: Optional[torch.Tensor] = None,
58 |     ):
59 |         """Training (actions given) -> loss dict. Eval (no actions) -> a_hat tensor."""
60 |         if actions is not None:
61 |             assert is_pad is not None, "is_pad required when actions is given"
62 |             a_hat, logvar = self.model(image, qpos, actions, is_pad)
63 |             return self._compute_loss(a_hat, actions, is_pad, mu, logvar)
64 |         a_hat, _ = self.model(image, qpos)
65 |         return a_hat

```

- If actions are supplied, the method assumes training/evaluation with labels and requires is_pad.
- The CVAE returns predicted actions plus mu/logvar, then _compute_loss turns them into scalar losses.
- If actions are absent, the model runs inference mode and returns the predicted action chunk directly.

model/act_policy.py:67-81 - Masked L1 plus KL loss

```

67 |     def _compute_loss(
68 |         self,
69 |         a_hat: torch.Tensor,

```

```

70 |         actions: torch.Tensor,
71 |         is_pad: torch.Tensor,
72 |         mu: torch.Tensor,
73 |         logvar: torch.Tensor,
74 |     ) -> dict:
75 |         l1 = F.l1_loss(a_hat, actions, reduction="none") # (B, k, action_dim)
76 |         l1 = (l1 * (~is_pad).unsqueeze(-1)).mean()
77 |
78 |         kl = -0.5 * (1 + logvar - mu.pow(2) - logvar.exp()).sum(dim=-1).mean()
79 |
80 |         total = l1 + self.kl_weight * kl
81 |         return {"loss": total, "l1": l1.detach(), "kl": kl.detach()}

```

- F.l1_loss with reduction="none" keeps per-batch, per-time, per-action errors.
- ~is_pad masks out padded future timesteps so trial-end padding does not train the model.
- The KL expression is the closed-form KL divergence from $N(\mu, \exp(\logvar))$ to $N(0, I)$.
- The returned l1 and kl are detached so metric logging does not keep extra computation graph references.

model/act_policy.py:87-119 - Numpy inference and image preprocessing

```

87 |     @torch.no_grad()
88 |     def inference(
89 |         self,
90 |         image_np: np.ndarray, # (H, W, 3) uint8 RGB
91 |         qpos_np: np.ndarray, # (state_dim,) float, raw Sensapex counts
92 |     ) -> np.ndarray:
93 |         """Rollout entry point. Returns (chunk_size, action_dim) in absolute units.
94 |
95 |         Callers feeding cv2-decoded frames must convert BGR -> RGB first; the
96 |         training dataset loads with PIL which is RGB.
97 |         """
98 |         self.eval()
99 |         device = self.qpos_mean.device
100 |
101 |         img = self._preprocess_image(image_np).to(device).unsqueeze(0) # (1, num_cam, 3, H, W)
102 |         qpos = torch.from_numpy(qpos_np.astype(np.float32)).to(device)
103 |         qpos = ((qpos - self.qpos_mean) / self.qpos_std).unsqueeze(0) # (1, state_dim)
104 |
105 |         a_hat, _ = self.model(img, qpos) # (1, k, action_dim)
106 |         a = a_hat[0] * self.action_std + self.action_mean
107 |         return a.cpu().numpy().astype(np.float32)
108 |
109 |     def _preprocess_image(self, img_np: np.ndarray) -> torch.Tensor:
110 |         h, w = img_np.shape[:2]
111 |         if (h, w) != (C.IMAGE_HEIGHT, C.IMAGE_WIDTH):
112 |             pil = Image.fromarray(img_np).resize(
113 |                 (C.IMAGE_WIDTH, C.IMAGE_HEIGHT), Image.BILINEAR
114 |             )
115 |             img_np = np.array(pil) # copy so torch can own it
116 |             x = torch.from_numpy(img_np).float() / 255.0 # (H, W, 3)
117 |             x = x.permute(2, 0, 1) # (3, H, W)
118 |             x = (x - self.image_mean) / self.image_std
119 |             return x.unsqueeze(0) # (num_cam=1, 3, H, W)

```

- @torch.no_grad disables gradient tracking for deployment.
- self.eval() puts dropout and frozen backbones in evaluation mode.
- The raw RGB image is resized/normalized just like training images; qpos is normalized with stored qpos stats.
- The CVAE predicts normalized actions, which are denormalized back into absolute robot target units before returning numpy float32.
- _preprocess_image adds the num_camera dimension expected by the model.

model/act_policy.py:124-133 - build_policy factory

```

124 | def build_policy(
125 |     stats: Optional[dict] = None,
126 |     stats_path: Path = C.STATS_PATH,
127 |     kl_weight: float = C.KL_WEIGHT,
128 |     **cvae_kwargs,
129 | ) -> ACTPolicy:
130 |     if stats is None:
131 |         with open(stats_path, "rb") as f:
132 |             stats = pickle.load(f)
133 |     return ACTPolicy(stats=stats, kl_weight=kl_weight, **cvae_kwargs)

```

- If stats are not passed directly, they are loaded from the configured pickle path.
- All remaining keyword arguments pass through to ACTCVAE, allowing train.py to select backbone/freeze/pretraining settings.

model/cvae.py

Purpose. Defines the actual ACT conditional variational autoencoder: style encoder during training, image/qpos/latent encoder-decoder transformer, learned queries, and action head.

Structure.

- reparameterize samples latent style z .
- `__init__` builds backbone, transformer, style encoder, projections, embeddings, and action head.
- `_encode_style` maps qpos/actions/is_pad to $\mu/\log\text{var}$.
- `_encode_image` converts camera images into transformer tokens.
- `forward` selects training or inference latent path and predicts an action chunk.

model/cvae.py:1-30 - Docstring, imports, and reparameterization

```
1 | """Conditional VAE action-chunking model for MicroACT.
2 |
3 | Two transformer stacks:
4 |
5 | - **Style encoder**: encoder-only over `[CLS, qpos_token, action_token_1..k]`.
6 |   The CLS output is projected to `(mu, logvar)`; a reparameterized latent
7 |   `z` is sampled. Only used during training.
8 |
9 | - **Main encoder-decoder**: encoder over `[latent_tok, qpos_tok, img_tokens]`,
10 |  decoder over `chunk_size` learned query embeddings, with a linear action
11 |  head mapping decoder outputs to `action_dim`.
12 |
13 | At inference the style encoder is skipped and `z = 0` (the prior mean).
14 | """
15 |
16 | from __future__ import annotations
17 |
18 | from typing import Optional, Tuple
19 |
20 | import torch
21 | import torch.nn as nn
22 |
23 | from config import config as C
24 | from model.backbone import build_backbone
25 | from model.transformer import build_encoder, build_transformer
26 |
27 |
28 | def reparameterize(mu: torch.Tensor, logvar: torch.Tensor) -> torch.Tensor:
29 |     std = (0.5 * logvar).exp()
30 |     return mu + torch.randn_like(std) * std
```

- The docstring describes the two transformer stacks: style encoder and main encoder-decoder.
- `reparameterize` implements $z = \mu + \epsilon * \text{std}$, where $\text{std} = \exp(0.5 * \log\text{var})$.
- This lets gradients flow through $\mu/\log\text{var}$ while sampling stochastic latent styles during training.

model/cvae.py:33-78 - ACTCVAE initialization

```
33 | class ACTCVAE(nn.Module):
34 |     def __init__(
35 |         self,
36 |         state_dim: int = C.STATE_DIM,
37 |         action_dim: int = C.ACTION_DIM,
38 |         hidden_dim: int = C.HIDDEN_DIM,
39 |         latent_dim: int = C.LATENT_DIM,
40 |         chunk_size: int = C.CHUNK_SIZE,
41 |         num_cameras: int = C.NUM_CAMERAS,
42 |         pretrained_backbone: bool = C.BACKBONE_PRETRAINED,
43 |         backbone_name: str = None,
44 |         freeze_backbone: bool = True,
45 |     ):
46 |         super().__init__()
47 |         self.state_dim = state_dim
48 |         self.action_dim = action_dim
49 |         self.hidden_dim = hidden_dim
50 |         self.latent_dim = latent_dim
51 |         self.chunk_size = chunk_size
52 |         self.num_cameras = num_cameras
53 |
54 |         # ---- Backbones ----
55 |         self.backbone = build_backbone(
56 |             hidden_dim=hidden_dim,
57 |             pretrained=pretrained_backbone,
58 |             backbone_name=backbone_name,
59 |             freeze=freeze_backbone,
60 |         )
61 |         self.transformer = build_transformer(d_model=hidden_dim)
62 |         self.style_encoder = build_encoder(d_model=hidden_dim)
63 |
64 |         # ---- Style-encoder IO ----
65 |         self.cls_embed = nn.Embedding(1, hidden_dim)
66 |         self.style_qpos_proj = nn.Linear(state_dim, hidden_dim)
67 |         self.style_action_proj = nn.Linear(action_dim, hidden_dim)
```

```

68 |         self.style_pos_embed = nn.Embedding(1 + 1 + chunk_size, hidden_dim)
69 |         self.latent_proj = nn.Linear(hidden_dim, 2 * latent_dim)
70 |
71 |         # ---- Main-encoder non-image tokens ----
72 |         self.latent_to_src = nn.Linear(latent_dim, hidden_dim)
73 |         self.qpos_to_src = nn.Linear(state_dim, hidden_dim)
74 |         self.extra_src_pos = nn.Embedding(2, hidden_dim)
75 |
76 |         # ---- Decoder queries + action head ----
77 |         self.query_embed = nn.Embedding(chunk_size, hidden_dim)
78 |         self.action_head = nn.Linear(hidden_dim, action_dim)

```

- Constructor defaults come from config, but can be overridden by build_policy/build_cvae callers.
- The image backbone is built first; the main transformer and style encoder are separate stacks.
- Style encoder inputs are CLS token, qpos projection, action projection, positional embedding, and latent projection.
- Main encoder uses latent_to_src and qpos_to_src to turn non-image vectors into transformer tokens.
- query_embed contains one learned query per future action timestep; action_head maps decoder hidden states to action dimensions.

model/cvae.py:82-105 - _encode_style

```

82 |     def _encode_style(
83 |         self,
84 |         qpos: torch.Tensor,      # (B, state_dim)
85 |         actions: torch.Tensor,   # (B, k, action_dim)
86 |         is_pad: torch.Tensor,   # (B, k) bool
87 |     ) -> Tuple[torch.Tensor, torch.Tensor]:
88 |         B = qpos.size(0)
89 |         device = qpos.device
90 |
91 |         cls = self.cls_embed.weight.unsqueeze(0).expand(B, -1, -1)      # (B, 1, D)
92 |         qpos_tok = self.style_qpos_proj(qpos).unsqueeze(1)             # (B, 1, D)
93 |         act_toks = self.style_action_proj(actions)                     # (B, k, D)
94 |         seq = torch.cat([cls, qpos_tok, act_toks], dim=1)               # (B, 2+k, D)
95 |         seq = seq.permute(1, 0, 2).contiguous()                        # (2+k, B, D)
96 |
97 |         pos = self.style_pos_embed.weight.unsqueeze(1).expand(-1, B, -1) # (2+k, B, D)
98 |
99 |         always_valid = torch.zeros(B, 2, dtype=torch.bool, device=device)
100 |         pad_mask = torch.cat([always_valid, is_pad], dim=1)            # (B, 2+k)
101 |
102 |         out = self.style_encoder(seq, src_key_padding_mask=pad_mask, pos=pos)
103 |         cls_out = out[0]                                                # (B, D)
104 |         mu, logvar = self.latent_proj(cls_out).chunk(2, dim=-1)
105 |         return mu, logvar

```

- Batch size and device are taken from qpos to construct masks on the correct device.
- CLS, qpos, and action tokens are concatenated as [CLS, qpos, actions].
- The tensor is permuted from batch-first to sequence-first because transformer.py follows PyTorch MultiheadAttention convention.
- The padding mask marks only padded action tokens invalid; CLS and qpos are always valid.
- The style encoder output at CLS position summarizes the sequence and is projected into mu and logvar.

model/cvae.py:107-127 - _encode_image

```

107 |     def _encode_image(self, image: torch.Tensor) -> Tuple[torch.Tensor, torch.Tensor]:
108 |         # image: (B, num_cam, 3, H, W)
109 |         B, N = image.shape[:2]
110 |         flat = image.flatten(0, 1)                                     # (B*N, 3, H, W)
111 |         feat, pos = self.backbone(flat)
112 |         # Single-encoder backbones return 4D (B*N, D, Hp, Wp); dual-encoder
113 |         # backbones already return pre-flattened tokens (S, B*N, D).
114 |         if feat.dim() == 4:
115 |             D, Hp, Wp = feat.shape[1:]
116 |             feat = feat.view(B, N, D, Hp, Wp).permute(0, 2, 1, 3, 4).flatten(2)
117 |             pos = pos.view(B, N, D, Hp, Wp).permute(0, 2, 1, 3, 4).flatten(2)
118 |             feat = feat.permute(2, 0, 1).contiguous()                  # (N*Hp*Wp, B, D)
119 |             pos = pos.permute(2, 0, 1).contiguous()
120 |         else:
121 |             # feat: (S, B*N, D) - multi-camera reshape: each camera contributes
122 |             # S tokens; concat along the token dimension.
123 |             S, BN, D = feat.shape
124 |             if N > 1:
125 |                 feat = feat.view(S, B, N, D).permute(2, 0, 1, 3).reshape(N * S, B, D)
126 |                 pos = pos.view(S, B, N, D).permute(2, 0, 1, 3).reshape(N * S, B, D)
127 |             return feat, pos

```

- Input shape is (B, num_cam, 3, H, W). flatten(0,1) lets the backbone process B*num_cam images as a normal image batch.
- Single encoders return spatial feature maps, which are reshaped to combine cameras and spatial positions into sequence tokens.
- Dual encoders already return flattened tokens, so only multi-camera reshaping is needed when N > 1.
- The method returns image features and positional encodings in sequence-first shape (S_img, B, D).

model/cvae.py:131-159 - forward pass

```

131 |     def forward(
132 |         self,
133 |         image: torch.Tensor,          # (B, num_cam, 3, H, W)
134 |         qpos: torch.Tensor,           # (B, state_dim)
135 |         actions: Optional[torch.Tensor] = None, # (B, k, action_dim) train-only
136 |         is_pad: Optional[torch.Tensor] = None,  # (B, k) train-only
137 |     ) -> Tuple[torch.Tensor, Tuple[torch.Tensor, torch.Tensor]]:
138 |         B = qpos.size(0)
139 |         device = qpos.device
140 |
141 |         if actions is not None:
142 |             mu, logvar = self._encode_style(qpos, actions, is_pad)
143 |             z = reparameterize(mu, logvar)
144 |         else:
145 |             mu = torch.zeros(B, self.latent_dim, device=device)
146 |             logvar = torch.zeros(B, self.latent_dim, device=device)
147 |             z = torch.zeros(B, self.latent_dim, device=device)
148 |
149 |         img_feat, img_pos = self._encode_image(image)          # (S_img, B, D)
150 |         latent_tok = self.latent_to_src(z).unsqueeze(0)        # (1, B, D)
151 |         qpos_tok = self.qpos_to_src(qpos).unsqueeze(0)         # (1, B, D)
152 |         extra_pos = self.extra_src_pos.weight.unsqueeze(1).expand(-1, B, -1) # (2, B, D)
153 |
154 |         src = torch.cat([latent_tok, qpos_tok, img_feat], dim=0) # (2+S_img, B, D)
155 |         src_pos = torch.cat([extra_pos, img_pos], dim=0)
156 |
157 |         hs = self.transformer(src, src_pos, self.query_embed.weight) # (k, B, D)
158 |         a_hat = self.action_head(hs.transpose(0, 1))            # (B, k, action_dim)
159 |         return a_hat, (mu, logvar)

```

- When actions are present, the style encoder produces mu/logvar and sampled z. During inference, mu/logvar/z are zeros, selecting the prior mean.
- Image tokens come from _encode_image; latent and qpos are projected into source tokens.
- src is [latent token, qpos token, image tokens]; src_pos contains matching learned positions for latent/qpos and sine positions for image tokens.
- The main transformer decodes chunk_size learned queries into hidden states.
- The action head converts hidden states from (k, B, D) to (B, k, action_dim).

model/cvae.py:162-163 - build_cvae

```

162 | def build_cvae(**kwargs) -> ACTCVAE:
163 |     return ACTCVAE(**kwargs)

```

- A small factory used by visualization/export scripts so they do not need to import the class directly.

model/transformer.py

Purpose. Implements DETR-style transformer primitives used both by the main ACT policy and by the CVAE style encoder.

Structure.

- Helper functions clone layers, choose activation, and add positions.
- TransformerEncoderLayer is self-attention plus feedforward.
- TransformerEncoder stacks encoder layers.
- TransformerDecoderLayer is query self-attention, cross-attention to memory, and feedforward.
- TransformerDecoder stacks decoder layers.
- Transformer combines encoder and decoder for the main ACT source/query path.
- build_transformer and build_encoder are factories.

model/transformer.py:1-35 - Docstring, imports, and helpers

```

1 | """Transformer primitives for MicroACT.
2 |
3 | DETR-style encoder / decoder with position embeddings injected at every
4 | layer. These blocks are shared by:
5 | - the main ACT encoder-decoder (image + qpos + latent -> action chunk)
6 | - the CVAE style encoder in model/cvae.py (cls + qpos + actions -> latent)
7 |
8 | Convention: sequence-first tensors (L, B, D), matching torch.nn.MultiheadAttention.
9 | """
10 |
11 | from __future__ import annotations
12 |
13 | import copy

```

```

14 | from typing import Optional
15 |
16 | import torch
17 | import torch.nn as nn
18 | import torch.nn.functional as F
19 |
20 | from config import config as C
21 |
22 |
23 | def _clones(module: nn.Module, n: int) -> nn.ModuleList:
24 |     return nn.ModuleList([copy.deepcopy(module) for _ in range(n)])
25 |
26 |
27 | def _activation(name: str):
28 |     if name == "relu": return F.relu
29 |     if name == "gelu": return F.gelu
30 |     if name == "glu": return F.glu
31 |     raise ValueError(f"unsupported activation: {name}")
32 |
33 |
34 | def _with_pos(x: torch.Tensor, pos: Optional[torch.Tensor]) -> torch.Tensor:
35 |     return x if pos is None else x + pos

```

- The file uses sequence-first tensors (L, B, D), matching nn.MultiheadAttention default.
- _clones deep-copies a layer N times so each layer has independent parameters.
- _activation maps a string to a PyTorch functional activation.
- _with_pos adds position encodings when provided and leaves tensors unchanged otherwise.

model/transformer.py:42-73 - TransformerEncoderLayer

```

42 | class TransformerEncoderLayer(nn.Module):
43 |     def __init__(
44 |         self,
45 |         d_model: int,
46 |         nhead: int,
47 |         dim_feedforward: int,
48 |         dropout: float = 0.1,
49 |         activation: str = "relu",
50 |     ):
51 |         super().__init__()
52 |         self.self_attn = nn.MultiheadAttention(d_model, nhead, dropout=dropout)
53 |         self.linear1 = nn.Linear(d_model, dim_feedforward)
54 |         self.linear2 = nn.Linear(dim_feedforward, d_model)
55 |         self.norm1 = nn.LayerNorm(d_model)
56 |         self.norm2 = nn.LayerNorm(d_model)
57 |         self.dropout = nn.Dropout(dropout)
58 |         self.dropout1 = nn.Dropout(dropout)
59 |         self.dropout2 = nn.Dropout(dropout)
60 |         self.activation = _activation(activation)
61 |
62 |     def forward(
63 |         self,
64 |         src: torch.Tensor,
65 |         src_key_padding_mask: Optional[torch.Tensor] = None,
66 |         pos: Optional[torch.Tensor] = None,
67 |     ) -> torch.Tensor:
68 |         q = k = _with_pos(src, pos)
69 |         src2 = self.self_attn(q, k, src, key_padding_mask=src_key_padding_mask)[0]
70 |         src = self.norm1(src + self.dropout1(src2))
71 |         src2 = self.linear2(self.dropout(self.activation(self.linear1(src))))
72 |         src = self.norm2(src + self.dropout2(src2))
73 |         return src

```

- The layer contains one multi-head self-attention block and one feedforward MLP block.
- q and k get positional information, while the value remains the raw source tensor.
- Residual connections plus LayerNorm stabilize training.
- The feedforward block expands from d_model to dim_feedforward and projects back to d_model.

model/transformer.py:76-98 - TransformerEncoder stack

```

76 | class TransformerEncoder(nn.Module):
77 |     def __init__(
78 |         self,
79 |         layer: nn.Module,
80 |         num_layers: int,
81 |         norm: Optional[nn.LayerNorm] = None,
82 |     ):
83 |         super().__init__()
84 |         self.layers = _clones(layer, num_layers)
85 |         self.norm = norm
86 |
87 |     def forward(
88 |         self,
89 |         src: torch.Tensor,
90 |         src_key_padding_mask: Optional[torch.Tensor] = None,
91 |         pos: Optional[torch.Tensor] = None,

```

```

92 |         ) -> torch.Tensor:
93 |             out = src
94 |             for layer in self.layers:
95 |                 out = layer(out, src_key_padding_mask=src_key_padding_mask, pos=pos)
96 |             if self.norm is not None:
97 |                 out = self.norm(out)
98 |             return out

```

- The stack is a ModuleList of cloned encoder layers.
- forward passes the sequence through each layer in order.
- An optional final LayerNorm can be applied, used by the style encoder factory.

model/transformer.py:105-150 - TransformerDecoderLayer

```

105 | class TransformerDecoderLayer(nn.Module):
106 |     def __init__(
107 |         self,
108 |         d_model: int,
109 |         nhead: int,
110 |         dim_feedforward: int,
111 |         dropout: float = 0.1,
112 |         activation: str = "relu",
113 |     ):
114 |         super().__init__()
115 |         self.self_attn = nn.MultiheadAttention(d_model, nhead, dropout=dropout)
116 |         self.multihead_attn = nn.MultiheadAttention(d_model, nhead, dropout=dropout)
117 |         self.linear1 = nn.Linear(d_model, dim_feedforward)
118 |         self.linear2 = nn.Linear(dim_feedforward, d_model)
119 |         self.norm1 = nn.LayerNorm(d_model)
120 |         self.norm2 = nn.LayerNorm(d_model)
121 |         self.norm3 = nn.LayerNorm(d_model)
122 |         self.dropout = nn.Dropout(dropout)
123 |         self.dropout1 = nn.Dropout(dropout)
124 |         self.dropout2 = nn.Dropout(dropout)
125 |         self.dropout3 = nn.Dropout(dropout)
126 |         self.activation = _activation(activation)
127 |
128 |     def forward(
129 |         self,
130 |         tgt: torch.Tensor,
131 |         memory: torch.Tensor,
132 |         memory_key_padding_mask: Optional[torch.Tensor] = None,
133 |         pos: Optional[torch.Tensor] = None,
134 |         query_pos: Optional[torch.Tensor] = None,
135 |     ) -> torch.Tensor:
136 |         q = k = _with_pos(tgt, query_pos)
137 |         tgt2 = self.self_attn(q, k, tgt)[0]
138 |         tgt = self.norm1(tgt + self.dropout1(tgt2))
139 |
140 |         tgt2 = self.multihead_attn(
141 |             query=_with_pos(tgt, query_pos),
142 |             key=_with_pos(memory, pos),
143 |             value=memory,
144 |             key_padding_mask=memory_key_padding_mask,
145 |         )[0]
146 |         tgt = self.norm2(tgt + self.dropout2(tgt2))
147 |
148 |         tgt2 = self.linear2(self.dropout(self.activation(self.linear1(tgt))))
149 |         tgt = self.norm3(tgt + self.dropout3(tgt2))
150 |         return tgt

```

- Decoder self-attention lets action queries communicate with each other.
- Cross-attention uses query positions on tgt and source positions on memory, so learned action queries attend to image/qpos/latent tokens.
- The third block is a feedforward MLP, again with residuals and LayerNorm.
- memory_key_padding_mask can prevent attention to invalid source positions when needed.

model/transformer.py:153-183 - TransformerDecoder stack

```

153 | class TransformerDecoder(nn.Module):
154 |     def __init__(
155 |         self,
156 |         layer: nn.Module,
157 |         num_layers: int,
158 |         norm: Optional[nn.LayerNorm] = None,
159 |     ):
160 |         super().__init__()
161 |         self.layers = _clones(layer, num_layers)
162 |         self.norm = norm
163 |
164 |     def forward(
165 |         self,
166 |         tgt: torch.Tensor,
167 |         memory: torch.Tensor,
168 |         memory_key_padding_mask: Optional[torch.Tensor] = None,
169 |         pos: Optional[torch.Tensor] = None,
170 |         query_pos: Optional[torch.Tensor] = None,
171 |     ) -> torch.Tensor:
172 |         out = tgt

```

```

173 |         for layer in self.layers:
174 |             out = layer(
175 |                 out,
176 |                 memory,
177 |                 memory_key_padding_mask=memory_key_padding_mask,
178 |                 pos=pos,
179 |                 query_pos=query_pos,
180 |             )
181 |         if self.norm is not None:
182 |             out = self.norm(out)
183 |         return out

```

- The decoder repeatedly refines the target query sequence against the encoded source memory.
- A final LayerNorm is used in the main Transformer decoder.

model/transformer.py:190-245 - Full Transformer

```

190 | class Transformer(nn.Module):
191 |     def __init__(
192 |         self,
193 |         d_model: int = C.HIDDEN_DIM,
194 |         nhead: int = C.NHEAD,
195 |         num_encoder_layers: int = C.ENC_LAYERS,
196 |         num_decoder_layers: int = C.DEC_LAYERS,
197 |         dim_feedforward: int = C.DIM_FEEDFORWARD,
198 |         dropout: float = C.DROPOUT,
199 |         activation: str = "relu",
200 |     ):
201 |         super().__init__()
202 |         enc_layer = TransformerEncoderLayer(
203 |             d_model, nhead, dim_feedforward, dropout, activation
204 |         )
205 |         self.encoder = TransformerEncoder(enc_layer, num_encoder_layers)
206 |
207 |         dec_layer = TransformerDecoderLayer(
208 |             d_model, nhead, dim_feedforward, dropout, activation
209 |         )
210 |         self.decoder = TransformerDecoder(
211 |             dec_layer, num_decoder_layers, norm=nn.LayerNorm(d_model)
212 |         )
213 |
214 |         self.d_model = d_model
215 |         self.nhead = nhead
216 |         self._reset_parameters()
217 |
218 |     def _reset_parameters(self):
219 |         for p in self.parameters():
220 |             if p.dim() > 1:
221 |                 nn.init.xavier_uniform_(p)
222 |
223 |     def forward(
224 |         self,
225 |         src: torch.Tensor,                # (S, B, D)
226 |         pos_embed: torch.Tensor,          # (S, B, D)
227 |         query_embed: torch.Tensor,        # (Q, D) or (Q, B, D)
228 |         src_key_padding_mask: Optional[torch.Tensor] = None, # (B, S)
229 |     ) -> torch.Tensor:
230 |         """Returns decoder output (Q, B, D)."""
231 |         memory = self.encoder(
232 |             src, src_key_padding_mask=src_key_padding_mask, pos=pos_embed
233 |         )
234 |
235 |         if query_embed.dim() == 2:
236 |             query_embed = query_embed.unsqueeze(1).expand(-1, src.size(1), -1)
237 |         tgt = torch.zeros_like(query_embed)
238 |
239 |         return self.decoder(
240 |             tgt,
241 |             memory,
242 |             memory_key_padding_mask=src_key_padding_mask,
243 |             pos=pos_embed,
244 |             query_pos=query_embed,
245 |         )

```

- Constructor builds one encoder stack and one decoder stack using config defaults.
- `_reset_parameters` applies Xavier initialization to weight tensors with more than one dimension.
- `forward` first encodes `src` into memory.
- A 2D query embedding table is expanded across batch if needed.
- `tgt` starts as zeros; all action-query identity comes from `query_pos`, following DETR style.
- The decoder returns (Q, B, D), one hidden vector per future action query.

model/transformer.py:248-264 - Factory functions

```

248 | def build_transformer(**kwargs) -> Transformer:
249 |     return Transformer(**kwargs)
250 |

```

```

251 |
252 | def build_encoder(
253 |     d_model: int = C.HIDDEN_DIM,
254 |     nhead: int = C.NHEAD,
255 |     num_layers: int = C.ENC_LAYERS,
256 |     dim_feedforward: int = C.DIM_FEEDFORWARD,
257 |     dropout: float = C.DROPOUT,
258 |     activation: str = "relu",
259 | ) -> TransformerEncoder:
260 |     """Encoder-only stack, used by the CVAE style encoder."""
261 |     layer = TransformerEncoderLayer(
262 |         d_model, nhead, dim_feedforward, dropout, activation
263 |     )
264 |     return TransformerEncoder(layer, num_layers, norm=nn.LayerNorm(d_model))

```

- `build_transformer` creates the main encoder-decoder transformer.
- `build_encoder` creates an encoder-only stack for the style encoder, with final `LayerNorm`.

model/backbone.py

Purpose. Defines all image feature encoders and the wrapper that normalizes their outputs into transformer-ready tokens plus positional embeddings.

Structure.

- `FrozenBatchNorm2d` supports stable ResNet behavior.
- `ResNet18Backbone` extracts layer4 features.
- `DinoV2Backbone` loads DINOv2 patch-token features.
- `CellposeBackbone` loads a frozen Cellpose encoder.
- `PositionEmbeddingSine2D` creates DETR-style spatial encodings.
- `Backbone` dispatches single or dual encoders and projects outputs to `hidden_dim`.

model/backbone.py:1-57 - Docstring, imports, and backbone constants

```

1 | """Image backbone for MicroACT.
2 |
3 | Three backbone families are supported, selected by the `backbone_name`
4 | argument (or the `BACKBONE` config constant):
5 |
6 | - `resnet18` - ImageNet-pretrained ResNet18 with frozen BN.
7 |               Default. ~11M backbone params.
8 | - `dinov2_vits14` - DINOv2 ViT-S/14, frozen by default. ~22M.
9 | - `dinov2_vitb14` - DINOv2 ViT-B/14, frozen by default. ~87M.
10 | - `dinov2_vitl14` - DINOv2 ViT-L/14, frozen by default. ~304M.
11 |
12 | Plus a **dual encoder** mode for hybrid generalist + domain-specialist
13 | features:
14 |
15 | - `dinov2_vits14+cellpose` - DINOv2 ViT-S as the primary encoder
16 |                               for general scene understanding +
17 |                               Cellpose 3 cyto3 U-Net encoder as a
18 |                               cell-aware specialist. Both frozen.
19 |                               Tokens from each are concatenated and
20 |                               tagged with a learned type embedding.
21 | - `resnet18+cellpose` - same idea with ResNet18 as primary.
22 |
23 | Single-encoder paths return spatial features `(B, num_channels, Hp, Wp)`,
24 | which the `Backbone` wrapper projects to `hidden_dim`, position-embeds,
25 | and flattens to a token sequence.
26 |
27 | Dual-encoder paths run both encoders independently, project each to
28 | `hidden_dim`, add a per-encoder type embedding, position-embed each in
29 | its own normalized coordinate frame, then concatenate along the token
30 | sequence. Cellpose features are 2x2-avg-pooled before flattening to
31 | keep the token budget reasonable (240x320 -> 7x10 = 70 tokens for
32 | Cellpose, vs 17x23 = 391 for DINOv2).
33 |
34 | Multi-camera handling (concatenation along spatial dim, per-camera pos
35 | offsets, etc.) lives in the CVAE module, not here.
36 | """
37 |
38 | from __future__ import annotations
39 |
40 | import math
41 |
42 | import torch
43 | import torch.nn as nn
44 | import torch.nn.functional as F
45 | import torchvision
46 | from torchvision.models._utils import IntermediateLayerGetter
47 |
48 | from config import config as C

```

```

49 |
50 |
51 | _DINOv2_EMBED_DIMS = {
52 |     "dinov2_vits14": 384,
53 |     "dinov2_vitb14": 768,
54 |     "dinov2_vitl14": 1024,
55 |     "dinov2_vitg14": 1536,
56 | }
57 | _DINOv2_PATCH_SIZE = 14

```

- The docstring lists supported backbone strings and explains single vs dual encoder outputs.
- `_DINOv2_EMBED_DIMS` maps DINO variant names to their channel widths.
- `_DINOv2_PATCH_SIZE` records the ViT patch size used to derive spatial grids.

model/backbone.py:64-87 - FrozenBatchNorm2d

```

64 | class FrozenBatchNorm2d(nn.Module):
65 |     """BatchNorm2d with running stats and affine parameters frozen as buffers."""
66 |
67 |     def __init__(self, num_features: int, eps: float = 1e-5):
68 |         super().__init__()
69 |         self.eps = eps
70 |         self.register_buffer("weight", torch.ones(num_features))
71 |         self.register_buffer("bias", torch.zeros(num_features))
72 |         self.register_buffer("running_mean", torch.zeros(num_features))
73 |         self.register_buffer("running_var", torch.ones(num_features))
74 |
75 |     def _load_from_state_dict(self, state_dict, prefix, *args, **kwargs):
76 |         # torchvision's BN writes num_batches_tracked; drop it to avoid key errors.
77 |         state_dict.pop(prefix + "num_batches_tracked", None)
78 |         super()._load_from_state_dict(state_dict, prefix, *args, **kwargs)
79 |
80 |     def forward(self, x: torch.Tensor) -> torch.Tensor:
81 |         w = self.weight.reshape(1, -1, 1, 1)
82 |         b = self.bias.reshape(1, -1, 1, 1)
83 |         rm = self.running_mean.reshape(1, -1, 1, 1)
84 |         rv = self.running_var.reshape(1, -1, 1, 1)
85 |         scale = w * (rv + self.eps).rsqrt()
86 |         bias = b - rm * scale
87 |         return x * scale + bias

```

- BatchNorm state is registered as buffers, not trainable parameters.
- `_load_from_state_dict` drops `num_batches_tracked` because torchvision BatchNorm checkpoints include it but this frozen replacement does not need it.
- forward manually applies affine batch normalization using stored running mean and variance.
- This is useful for small batches where live BatchNorm statistics would be noisy.

model/backbone.py:94-109 - ResNet18Backbone

```

94 | class ResNet18Backbone(nn.Module):
95 |     def __init__(self, pretrained: bool = True):
96 |         super().__init__()
97 |         weights = (
98 |             torchvision.models.ResNet18_Weights.IMAGENET1K_V1 if pretrained else None
99 |         )
100 |         resnet = torchvision.models.resnet18(
101 |             weights=weights,
102 |             norm_layer=FrozenBatchNorm2d,
103 |         )
104 |         self.body = IntermediateLayerGetter(resnet, return_layers={"layer4": "feat"})
105 |         self.num_channels = 512
106 |
107 |     def forward(self, x: torch.Tensor) -> torch.Tensor:
108 |         # (B, 3, H, W) -> (B, 512, H/32, W/32)
109 |         return self.body(x)["feat"]

```

- The constructor optionally chooses ImageNet weights.
- torchvision ResNet18 is built with FrozenBatchNorm2d.
- IntermediateLayerGetter returns only layer4, the final spatial feature map.
- forward returns a tensor shaped approximately (B, 512, H/32, W/32).

model/backbone.py:116-167 - DinoV2Backbone

```

116 | class DinoV2Backbone(nn.Module):
117 |     """DINOv2 ViT backbone, frozen by default.
118 |
119 |     The model expects spatial dims divisible by the patch size (14). We
120 |     bilinearly resize on the fly, so the rest of the pipeline is free to
121 |     use any image shape (we currently use 240x320 -> resized to 238x322 =
122 |     17x23 patches).
123 |     """
124 |
125 |     def __init__(self, name: str = "dinov2_vits14", freeze: bool = True):

```

```

126 |         super().__init__()
127 |         if name not in _DINOv2_EMBED_DIMS:
128 |             raise ValueError(
129 |                 f"Unknown DINOv2 variant {name!r}. "
130 |                 f"Choose from {list(_DINOv2_EMBED_DIMS)}."
131 |             )
132 |         self.name = name
133 |         self.num_channels = _DINOv2_EMBED_DIMS[name]
134 |         self.patch_size = _DINOv2_PATCH_SIZE
135 |
136 |         # First call downloads ~85 MB (ViT-S) -> ~1 GB (ViT-L) into ~/.cache/torch/hub.
137 |         self.dinov2 = torch.hub.load("facebookresearch/dinov2", name, verbose=False)
138 |
139 |         self.frozen = freeze
140 |         if freeze:
141 |             for p in self.dinov2.parameters():
142 |                 p.requires_grad = False
143 |
144 |     def train(self, mode: bool = True):
145 |         # Frozen DINOv2 must stay in eval mode to disable any train-only ops.
146 |         super().train(mode)
147 |         if self.frozen:
148 |             self.dinov2.eval()
149 |         return self
150 |
151 |     def forward(self, x: torch.Tensor) -> torch.Tensor:
152 |         # x: (B, 3, H, W). Resize to nearest multiple of patch_size (round down).
153 |         B, _, H, W = x.shape
154 |         Hp_full = max(self.patch_size, (H // self.patch_size) * self.patch_size)
155 |         Wp_full = max(self.patch_size, (W // self.patch_size) * self.patch_size)
156 |         if (Hp_full, Wp_full) != (H, W):
157 |             x = F.interpolate(x, size=(Hp_full, Wp_full), mode="bilinear", align_corners=False)
158 |         Hp, Wp = Hp_full // self.patch_size, Wp_full // self.patch_size
159 |
160 |         # forward_features returns a dict with 'x_norm_patchtokens' shape (B, N, D)
161 |         # where N = Hp * Wp. Skip the CLS token (we use the spatial grid directly).
162 |         ctx = torch.no_grad() if self.frozen else torch.enable_grad()
163 |         with ctx:
164 |             out = self.dinov2.forward_features(x)
165 |             tokens = out["x_norm_patchtokens"] # (B, N, D)
166 |             feat = tokens.transpose(1, 2).reshape(B, self.num_channels, Hp, Wp)
167 |             return feat

```

- The constructor validates the model name and loads weights through torch.hub.
- If freeze=True, all DINOv2 parameters get requires_grad=False.
- train() is overridden so frozen DINOv2 stays in eval mode even when the parent model enters training mode.
- forward resizes images down to dimensions divisible by 14, runs DINOv2 forward_features, takes x_norm_patchtokens, and reshapes tokens to a spatial feature map.

model/backbone.py:174-247 - CellposeBackbone

```

174 | _CELLPOSE_WEIGHTS_URL = "https://www.cellpose.org/models/cyto3"
175 | _CELLPOSE_NBASE = [2, 32, 64, 128, 256] # cyto3 default U-Net channel widths
176 |
177 |
178 | class CellposeBackbone(nn.Module):
179 |     """Cellpose 3 cyto3 encoder as a frozen feature extractor.
180 |
181 |     Discards the U-Net's decoder + segmentation heads; uses only the
182 |     deepest encoder feature map at 1/8 resolution, 256 channels. For
183 |     240x320 RGB input -> output is (B, 256, 30, 40).
184 |
185 |     The model expects 2-channel input (cyto, nuclei). We map RGB to a
186 |     single luminance channel and pad the second channel with zeros -
187 |     fine for microscope footage with no nuclear stain.
188 |
189 |     First instantiation downloads ~25 MB of cyto3 weights into
190 |     ~/.cellpose/models/cyto3.
191 |     """
192 |
193 |     def __init__(self, freeze: bool = True):
194 |         super().__init__()
195 |         # Lazy import: cellpose's package __init__ pulls numba via
196 |         # cellpose.dynamics, which has a known coverage-package conflict.
197 |         # The resnet_torch module is pure PyTorch - import only that.
198 |         from cellpose.resnet_torch import CPnet
199 |
200 |         self.net = CPnet(nbase=_CELLPOSE_NBASE, nout=3, sz=3, mkldnn=False)
201 |         self._load_pretrained()
202 |         self.num_channels = _CELLPOSE_NBASE[-1] # 256
203 |
204 |         self.frozen = freeze
205 |         if freeze:
206 |             for p in self.net.parameters():
207 |                 p.requires_grad = False
208 |
209 |         # ImageNet luminance weights, registered as buffers so .to(device)
210 |         # moves them with the model.
211 |         self.register_buffer(
212 |             "_luma_w",

```

```

213 |         torch.tensor([0.299, 0.587, 0.114]).view(1, 3, 1, 1),
214 |         persistent=False,
215 |     )
216 |
217 |     def _load_pretrained(self):
218 |         import pathlib
219 |         cache = pathlib.Path.home() / ".cellpose" / "models" / "cyto3"
220 |         cache.parent.mkdir(parents=True, exist_ok=True)
221 |         if not cache.exists():
222 |             torch.hub.download_url_to_file(
223 |                 _CELLPOSE_WEIGHTS_URL, str(cache), progress=False
224 |             )
225 |             state = torch.load(cache, map_location="cpu", weights_only=True)
226 |             self.net.load_state_dict(state, strict=False)
227 |
228 |     def train(self, mode: bool = True):
229 |         # Frozen Cellpose stays in eval mode (BN running stats fixed, etc.)
230 |         super().train(mode)
231 |         if self.frozen:
232 |             self.net.eval()
233 |         return self
234 |
235 |     def _to_2chan(self, x: torch.Tensor) -> torch.Tensor:
236 |         """RGB (B,3,H,W) -> 2-channel (luminance, zero) for Cellpose."""
237 |         gray = (x * self._luma_w).sum(dim=1, keepdim=True) # (B, 1, H, W)
238 |         zero = torch.zeros_like(gray)
239 |         return torch.cat([gray, zero], dim=1) # (B, 2, H, W)
240 |
241 |     def forward(self, x: torch.Tensor) -> torch.Tensor:
242 |         # x: (B, 3, H, W) ImageNet-normalized RGB.
243 |         x = self._to_2chan(x)
244 |         ctx = torch.no_grad() if self.frozen else torch.enable_grad()
245 |         with ctx:
246 |             feats = self.net.downsample(x)
247 |         return feats[-1] # (B, 256, H/8, W/8)

```

- Cellpose weights URL and channel widths are declared at module level.
- The constructor lazily imports CPnet to avoid broader Cellpose import side effects.
- `_load_pretrained` downloads or loads cyto3 weights from the user cache and loads them non-strictly.
- `_luma_w` converts RGB to luminance as a registered non-persistent buffer, so it moves with device but is not saved in checkpoints.
- `_to_2chan` maps RGB to Cellpose expected two-channel input: luminance plus zeros.
- `forward` returns the deepest downsample feature map, shaped about (B, 256, H/8, W/8).

model/backbone.py:254-283 - PositionEmbeddingSine2D

```

254 | class PositionEmbeddingSine2D(nn.Module):
255 |     """Returns a (B, 2*num_pos_feats, H, W) position embedding for a feature map."""
256 |
257 |     def __init__(self, num_pos_feats: int = 128, temperature: int = 10000):
258 |         super().__init__()
259 |         self.num_pos_feats = num_pos_feats
260 |         self.temperature = temperature
261 |         self.scale = 2 * math.pi
262 |
263 |     def forward(self, feat: torch.Tensor) -> torch.Tensor:
264 |         b, _, h, w = feat.shape
265 |         device = feat.device
266 |
267 |         # All positions are "valid" (no padding mask in our image inputs).
268 |         ones = torch.ones((b, h, w), device=device)
269 |         y_embed = ones.cumsum(1, dtype=torch.float32)
270 |         x_embed = ones.cumsum(2, dtype=torch.float32)
271 |
272 |         eps = 1e-6
273 |         y_embed = y_embed / (y_embed[:, -1:, :] + eps) * self.scale
274 |         x_embed = x_embed / (x_embed[:, :, -1:] + eps) * self.scale
275 |
276 |         dim_t = torch.arange(self.num_pos_feats, dtype=torch.float32, device=device)
277 |         dim_t = self.temperature ** (2 * (dim_t // 2) / self.num_pos_feats)
278 |
279 |         pos_x = x_embed[:, :, :, None] / dim_t
280 |         pos_y = y_embed[:, :, :, None] / dim_t
281 |         pos_x = torch.stack((pos_x[..., 0::2].sin(), pos_x[..., 1::2].cos()), dim=4).flatten(3)
282 |         pos_y = torch.stack((pos_y[..., 0::2].sin(), pos_y[..., 1::2].cos()), dim=4).flatten(3)
283 |         return torch.cat((pos_y, pos_x), dim=3).permute(0, 3, 1, 2)

```

- This creates deterministic 2D sine/cosine position encodings, similar to DETR.
- `cumsum` creates normalized x and y coordinate grids.
- Coordinates are scaled to $0..2\pi$, divided by frequency terms, then transformed by sin/cos.
- The returned tensor is channel-first (B, D, H, W), matching image features.

model/backbone.py:290-304 - `_build_single_encoder`

```

290 | def _build_single_encoder(name: str, pretrained: bool, freeze: bool):

```

```

291 |     """Returns (module, num_channels) for a single backbone name."""
292 |     if name == "resnet18":
293 |         m = ResNet18Backbone(pretrained=pretrained)
294 |         return m, m.num_channels
295 |     if name in _DINOv2_EMBED_DIMS:
296 |         m = DinoV2Backbone(name=name, freeze=freeze)
297 |         return m, m.num_channels
298 |     if name == "cellpose":
299 |         m = CellposeBackbone(freeze=freeze)
300 |         return m, m.num_channels
301 |     raise ValueError(
302 |         f"Unknown backbone {name!r}. Supported: 'resnet18', "
303 |         f"{list(_DINOv2_EMBED_DIMS)}, 'cellpose'."
304 |     )

```

- This dispatcher converts a string name into a concrete encoder module plus its output channel count.
- Unsupported names raise a clear `ValueError`.

model/backbone.py:307-362 - Backbone initialization

```

307 | class Backbone(nn.Module):
308 |     """Wraps one or two image feature extractors + projection + pos embed.
309 |
310 |     Single-encoder mode (e.g. `backbone_name='resnet18'` or
311 |     `dinov2_vits14`):
312 |         forward(x) returns (feat, pos) - both shape (B, hidden_dim, Hp, Wp).
313 |         Backward-compatible with previous Backbone.
314 |
315 |     Dual-encoder mode (e.g. `backbone_name='dinov2_vits14+cellpose'`):
316 |         forward(x) returns (tokens, pos_tokens) - both shape (S, B, hidden_dim).
317 |         Tokens from the two encoders are concatenated along the sequence
318 |         dim, each tagged with a learned type embedding. Cellpose features
319 |         are 2x2-avg-pooled before flattening to keep the token budget down.
320 |
321 |     """
322 |     def __init__(
323 |         self,
324 |         hidden_dim: int = C.HIDDEN_DIM,
325 |         pretrained: bool = C.BACKBONE_PRETRAINED,
326 |         backbone_name: str = None,
327 |         freeze: bool = True,
328 |     ):
329 |         super().__init__()
330 |         backbone_name = backbone_name or getattr(C, "BACKBONE", "resnet18")
331 |         self.backbone_name = backbone_name
332 |         self.hidden_dim = hidden_dim
333 |
334 |         parts = backbone_name.split("+")
335 |         self.is_dual = len(parts) > 1
336 |         if self.is_dual and len(parts) != 2:
337 |             raise ValueError(
338 |                 f"Only 2-encoder fusion supported, got {len(parts)} parts in "
339 |                 f"{backbone_name!r}"
340 |             )
341 |
342 |         # ---- Build encoder(s) ----
343 |         primary_name = parts[0]
344 |         primary, primary_chan = _build_single_encoder(primary_name, pretrained, freeze)
345 |         self._set_encoder(primary_name, primary)
346 |
347 |         if self.is_dual:
348 |             aux_name = parts[1]
349 |             aux, aux_chan = _build_single_encoder(aux_name, pretrained, freeze)
350 |             self._set_encoder(aux_name, aux)
351 |             self.input_proj_aux = nn.Conv2d(aux_chan, hidden_dim, kernel_size=1)
352 |             # 2x2 average pool to halve aux spatial extent (Cellpose is at
353 |             # 1/8 native resolution; this brings it to 1/16, similar to
354 |             # DINOv2's 1/14 patch grid).
355 |             self.aux_pool = nn.AvgPool2d(kernel_size=2, stride=2)
356 |             # Token-type embeddings: 0 = primary, 1 = auxiliary.
357 |             self.type_embed = nn.Embedding(2, hidden_dim)
358 |             self.primary_name = primary_name
359 |             self.aux_name = aux_name
360 |
361 |             self.input_proj = nn.Conv2d(primary_chan, hidden_dim, kernel_size=1)
362 |             self.pos_embed = PositionEmbeddingSine2D(num_pos_feats=hidden_dim // 2)

```

- `backbone_name` can be a single encoder like `resnet18` or a dual string like `dinov2_vits14+cellpose`.
- The code only permits two-part dual fusion.
- The primary encoder is always built and assigned through `_set_encoder`.
- For dual mode, the auxiliary encoder gets its own 1x1 projection, an average pool, and a learned type embedding that distinguishes primary vs auxiliary tokens.
- `input_proj` maps primary channels into transformer `hidden_dim`; `pos_embed` generates spatial encodings at `hidden_dim`.

model/backbone.py:364-384 - Encoder attribute routing

```

364 |     def _set_encoder(self, name: str, module: nn.Module) -> None:

```

```

365 |         # Use a name-specific attribute so existing ResNet/DINOv2 checkpoints
366 |         # still load (their state_dict keys reference 'resnet'/'dinov2').
367 |         if name == "resnet18":
368 |             self.resnet = module
369 |         elif name in _DINOv2_EMBED_DIMS:
370 |             self.dinov2 = module
371 |         elif name == "cellpose":
372 |             self.cellpose = module
373 |         else:
374 |             raise ValueError(f"unreachable: {name!r}")
375 |
376 |     def _primary_feat(self, x: torch.Tensor) -> torch.Tensor:
377 |         primary_name = self.primary_name if self.is_dual else self.backbone_name
378 |         if primary_name == "resnet18":
379 |             return self.resnet(x)
380 |         if primary_name in _DINOv2_EMBED_DIMS:
381 |             return self.dinov2(x)
382 |         if primary_name == "cellpose":
383 |             return self.cellpose(x)
384 |         raise ValueError(f"unreachable: {primary_name!r}")

```

- `_set_encoder` gives modules stable attribute names such as `resnet`, `dinov2`, or `cellpose`.
- Stable names help older checkpoints load because `state_dict` keys stay predictable.
- `_primary_feat` selects the right module to call for the primary encoder.

model/backbone.py:386-420 - Backbone forward

```

386 |     def forward(self, x: torch.Tensor):
387 |         """
388 |         Single mode -> returns (feat, pos), both (B, hidden_dim, Hp, Wp).
389 |         Dual mode -> returns (tokens, pos_tokens), both (S, B, hidden_dim).
390 |         """
391 |         # Primary encoder
392 |         feat_p = self._primary_feat(x)
393 |         feat_p = self.input_proj(feat_p)
394 |         pos_p = self.pos_embed(feat_p)
395 |
396 |         if not self.is_dual:
397 |             return feat_p, pos_p
398 |
399 |         # Auxiliary encoder (currently always Cellpose)
400 |         feat_a = self.cellpose(x) # (B, 256, H/8, W/8)
401 |         feat_a = self.aux_pool(feat_a) # (B, 256, H/16, W/16)
402 |         feat_a = self.input_proj_aux(feat_a) # (B, D, h, w)
403 |         pos_a = self.pos_embed(feat_a)
404 |
405 |         # Add type embeddings (broadcast over spatial dims).
406 |         D = self.hidden_dim
407 |         type_p = self.type_embed.weight[0].view(1, D, 1, 1)
408 |         type_a = self.type_embed.weight[1].view(1, D, 1, 1)
409 |         feat_p = feat_p + type_p
410 |         feat_a = feat_a + type_a
411 |
412 |         # Flatten each to (S, B, D) and concat along token dim.
413 |         f_p = feat_p.flatten(2).permute(2, 0, 1) # (S_p, B, D)
414 |         f_a = feat_a.flatten(2).permute(2, 0, 1) # (S_a, B, D)
415 |         p_p = pos_p.flatten(2).permute(2, 0, 1)
416 |         p_a = pos_a.flatten(2).permute(2, 0, 1)
417 |
418 |         tokens = torch.cat([f_p, f_a], dim=0) # (S_p + S_a, B, D)
419 |         pos = torch.cat([p_p, p_a], dim=0)
420 |         return tokens, pos

```

- The primary encoder produces features, then `input_proj` converts channels to `hidden_dim` and `pos_embed` creates matching spatial positions.
- Single mode returns feature and position maps directly; `cvae.py` later flattens them into tokens.
- Dual mode additionally runs `Cellpose`, pools its feature map, projects it, and position-embeds it.
- Learned type embeddings are added to feature maps so the transformer can distinguish primary and auxiliary streams.
- Both streams are flattened into sequence-first tokens and concatenated along the sequence dimension.

model/backbone.py:423-434 - build_backbone

```

423 |     def build_backbone(
424 |         hidden_dim: int = C.HIDDEN_DIM,
425 |         pretrained: bool = C.BACKBONE_PRETRAINED,
426 |         backbone_name: str = None,
427 |         freeze: bool = True,
428 |     ) -> Backbone:
429 |         return Backbone(
430 |             hidden_dim=hidden_dim,
431 |             pretrained=pretrained,
432 |             backbone_name=backbone_name,
433 |             freeze=freeze,
434 |         )

```

- A small factory used by ACTCVAE to construct the wrapper without importing the class directly.

viz_summary.py

Purpose. Utility script that prints a readable module hierarchy and torchinfo summaries for inference and training paths.

Structure.

- Imports torchinfo.summary and build_cvae.
- main builds a dummy model and dummy tensors.
- It prints module hierarchy, then summary for inference and training signatures.
- CLI accepts backbone and summary depth.

viz_summary.py:1-22 - Docstring and imports

```
1 | """High-level layer summary of the ACT model.
2 |
3 | Prints a Keras-style table - every named layer, output shape, and param
4 | count, with no math operations. The right granularity for "what is this
5 | model made of?" without drowning in autograd ops.
6 |
7 | Two views are printed:
8 |   1. nn.Module hierarchy (just names, no shapes)
9 |   2. torchinfo.summary table (with shapes + params), at three depths
10 |
11 | Output goes to stdout. Pipe to a file if you want to keep it:
12 |   python viz_summary.py > architecture.txt
13 |   python viz_summary.py --backbone dinov2_vits14 > architecture_dinov2.txt
14 | """
15 |
16 | import argparse
17 |
18 | import torch
19 | from torchinfo import summary
20 |
21 | from config import config as C
22 | from model.cvae import build_cvae
```

- The docstring explains that this is a high-level shape/parameter summary tool, not a training script.
- torchinfo.summary runs forward passes and records module output shapes/parameter counts.
- build_cvae constructs the same core model used by training and export scripts.

viz_summary.py:25-39 - Build model and dummy inputs

```
25 | def main(depth: int = 1, backbone: str = None) -> None:
26 |     cvae = build_cvae(backbone_name=backbone).eval()
27 |     print(f"backbone: {cvae.backbone.backbone_name}")
28 |
29 |     print("=" * 78)
30 |     print("nn.Module hierarchy (names only)")
31 |     print("=" * 78)
32 |     print(cvae)
33 |     print()
34 |
35 |     # Two forward calls - inference path (no actions) and training path (with).
36 |     image = torch.zeros(1, C.NUM_CAMERAS, 3, C.IMAGE_HEIGHT, C.IMAGE_WIDTH)
37 |     qpos = torch.zeros(1, C.STATE_DIM)
38 |     actions = torch.zeros(1, C.CHUNK_SIZE, C.ACTION_DIM)
39 |     is_pad = torch.zeros(1, C.CHUNK_SIZE, dtype=torch.bool)
```

- main builds an ACTCVAE for the requested backbone and switches to eval mode.
- It prints the raw nn.Module hierarchy, which is useful before looking at tensor shapes.
- Dummy image/qpos/action/is_pad tensors match configured dimensions so summary can trace both forward paths.

viz_summary.py:41-62 - torchinfo summaries

```
41 |     print("=" * 78)
42 |     print(f"torchinfo.summary - INFERENCE path (depth={depth})")
43 |     print("=" * 78)
44 |     summary(
45 |         cvae,
46 |         input_data=(image, qpos),
47 |         depth=depth,
48 |         col_names=("input_size", "output_size", "num_params"),
49 |         verbose=1,
50 |     )
51 |     print()
52 |
53 |     print("=" * 78)
54 |     print(f"torchinfo.summary - TRAINING path (depth={depth})")
55 |     print("=" * 78)
56 |     summary(
```

```

57 |         cvae,
58 |         input_data=(image, qpos, actions, is_pad),
59 |         depth=depth,
60 |         col_names=("input_size", "output_size", "num_params"),
61 |         verbose=1,
62 |     )

```

- The first summary calls `cvae(image, qpos)`, exercising inference mode without actions.
- The second summary calls `cvae(image, qpos, actions, is_pad)`, exercising training mode with the style encoder.
- `depth` controls how many nested modules `torchinfo` expands.

`viz_summary.py:65-73` - CLI entry point

```

65 | if __name__ == "__main__":
66 |     p = argparse.ArgumentParser()
67 |     p.add_argument("--backbone", type=str, default=None,
68 |                   help="resnet18 | dinov2_vits14 | dinov2_vitb14 | dinov2_vitl14. "
69 |                       "Defaults to config.BACKBONE.")
70 |     p.add_argument("--depth", type=int, default=1,
71 |                   help="torchinfo nesting depth. 1 = top-level only, 4+ = inner attention.")
72 |     args = p.parse_args()
73 |     main(depth=args.depth, backbone=args.backbone)

```

- The script supports `--backbone` and `--depth`, then passes them to `main`.
- This lets a reader compare model variants without modifying source.

export_onnx.py

Purpose. Exports two ONNX graphs for architecture inspection in Netron: deployment inference and training with style encoder.

Structure.

- Wrapper classes expose clean forward signatures for ONNX.
- `main` builds `ACTCVAE`, creates dummy inputs, and calls `torch.onnx.export` twice.
- CLI controls backbone and whether frozen backbone internals are included.

`export_onnx.py:1-21` - Docstring and imports

```

1 | """Export the ACT model to ONNX for visualization in Netron.
2 |
3 | Produces two files under `onnx_exports/<backbone>`:
4 | - act_inference.onnx : forward path used at deployment (image + qpos -> action chunk).
5 |                       No CVAE style encoder.
6 | - act_training.onnx  : forward path used during training (image + qpos + actions + is_pad
7 |                       -> action chunk + mu + logvar). Includes the style encoder.
8 |
9 | After running, open https://netron.app in your browser and drag the .onnx files in.
10 |
11 | Use `--backbone` to switch between ResNet18 and DINOv2 variants.
12 | """
13 |
14 | import argparse
15 | from pathlib import Path
16 |
17 | import torch
18 | import torch.nn as nn
19 |
20 | from config import config as C
21 | from model.cvae import build_cvae

```

- The docstring names the two exported files and what path each represents.
- `torch.onnx.export` needs concrete example tensors and an `nn.Module` forward signature.
- `config` supplies dummy input shapes.

`export_onnx.py:24-51` - Inference and training wrappers

```

24 | class _InferenceWrapper(nn.Module):
25 |     """Inference path: no actions, no is_pad. Returns predicted action chunk."""
26 |
27 |     def __init__(self, cvae: nn.Module):
28 |         super().__init__()
29 |         self.cvae = cvae
30 |
31 |     def forward(self, image: torch.Tensor, qpos: torch.Tensor) -> torch.Tensor:
32 |         a_hat, _ = self.cvae(image, qpos, actions=None, is_pad=None)
33 |         return a_hat
34 |
35 |

```

```

36 | class _TrainingWrapper(nn.Module):
37 |     """Training path: actions + is_pad provided. Returns chunk + style distribution."""
38 |
39 |     def __init__(self, cvae: nn.Module):
40 |         super().__init__()
41 |         self.cvae = cvae
42 |
43 |     def forward(
44 |         self,
45 |         image: torch.Tensor,
46 |         qpos: torch.Tensor,
47 |         actions: torch.Tensor,
48 |         is_pad: torch.Tensor,
49 |     ):
50 |         a_hat, (mu, logvar) = self.cvae(image, qpos, actions=actions, is_pad=is_pad)
51 |         return a_hat, mu, logvar

```

- _InferenceWrapper hides the optional actions/is_pad arguments and returns only action chunks.
- _TrainingWrapper exposes image, qpos, actions, and is_pad, then returns action_chunk, mu, and logvar.
- These wrappers make the exported ONNX graphs cleaner and easier to inspect.

export_onnx.py:54-74 - main setup and dummy inputs

```

54 | def main(
55 |     out_dir: Path = Path("onnx_exports"),
56 |     opset: int = 18,
57 |     backbone: str = None,
58 |     freeze_backbone: bool = True,
59 | ) -> None:
60 |     cvae = build_cvae(backbone_name=backbone, freeze_backbone=freeze_backbone).eval()
61 |     backbone_name = cvae.backbone.backbone_name
62 |     out_dir = out_dir / backbone_name
63 |     out_dir.mkdir(parents=True, exist_ok=True)
64 |
65 |     print(f"built cvae: {sum(p.numel() for p in cvae.parameters()) / 1e6:.1f} M params "
66 |           f"(backbone={backbone_name})")
67 |
68 |     image = torch.zeros(1, C.NUM_CAMERAS, 3, C.IMAGE_HEIGHT, C.IMAGE_WIDTH)
69 |     qpos = torch.zeros(1, C.STATE_DIM)
70 |     actions = torch.zeros(1, C.CHUNK_SIZE, C.ACTION_DIM)
71 |     is_pad = torch.zeros(1, C.CHUNK_SIZE, dtype=torch.bool)
72 |
73 |     inference_path = out_dir / "act_inference.onnx"
74 |     training_path = out_dir / "act_training.onnx"

```

- The CVAE is built and set to eval mode.
- Output goes under onnx_exports/<backbone>.
- Dummy tensors define the static non-batch dimensions used by ONNX export.

export_onnx.py:76-108 - ONNX export calls

```

76 |     print(f"exporting inference graph -> {inference_path}")
77 |     torch.onnx.export(
78 |         _InferenceWrapper(cvae),
79 |         (image, qpos),
80 |         inference_path.as_posix(),
81 |         input_names=["image", "qpos"],
82 |         output_names=["action_chunk"],
83 |         opset_version=opset,
84 |         dynamic_axes={
85 |             "image": {0: "batch"},
86 |             "qpos": {0: "batch"},
87 |             "action_chunk": {0: "batch"},
88 |         },
89 |     )
90 |
91 |     print(f"exporting training graph -> {training_path}")
92 |     torch.onnx.export(
93 |         _TrainingWrapper(cvae),
94 |         (image, qpos, actions, is_pad),
95 |         training_path.as_posix(),
96 |         input_names=["image", "qpos", "actions", "is_pad"],
97 |         output_names=["action_chunk", "mu", "logvar"],
98 |         opset_version=opset,
99 |         dynamic_axes={
100 |             "image": {0: "batch"},
101 |             "qpos": {0: "batch"},
102 |             "actions": {0: "batch"},
103 |             "is_pad": {0: "batch"},
104 |             "action_chunk": {0: "batch"},
105 |             "mu": {0: "batch"},
106 |             "logvar": {0: "batch"},
107 |         },
108 |     )

```

- The inference graph has inputs image and qpos, and output action_chunk.
- The training graph includes actions/is_pad and outputs action_chunk, mu, and logvar.

- `dynamic_axes` marks the batch dimension dynamic so the exported graph is not fixed to batch size 1.

export_onnx.py:110-123 - CLI entry point

```

110 |     print("done.")
111 |     print(f"open https://netron.app and drag in:\n  {inference_path}\n  {training_path}")
112 |
113 |
114 | if __name__ == "__main__":
115 |     p = argparse.ArgumentParser()
116 |     p.add_argument("--backbone", type=str, default=None,
117 |                   help="resnet18 | dinov2_vits14 | dinov2_vitb14 | dinov2_vitl14. "
118 |                       "Defaults to config.BACKBONE.")
119 |     p.add_argument("--unfreeze-backbone", action="store_true",
120 |                   help="DINOv2 backbones are frozen by default; pass this to include "
121 |                       "their internal ops in the exported graph.")
122 |     args = p.parse_args()
123 |     main(backbone=args.backbone, freeze_backbone=not args.unfreeze_backbone)

```

- The script prints final paths for Netron inspection.
- `--unfreeze-backbone` affects whether frozen backbone operations appear as part of the exported graph behavior.

viz_torchviz.py

Purpose. Renders PyTorch autograd graphs as SVGs. This is a debugging/inspection tool for gradient flow, not model training.

Structure.

- `render` wraps `torchviz.make_dot`.
- `main` builds model and dummy gradient-requiring tensors.
- It renders focused graphs for backbone and style encoder, then full inference/training graphs.
- CLI accepts backbone and unfreeze flag.

viz_torchviz.py:1-28 - Docstring and imports

```

1 | """Render the model's autograd graph with torchviz.
2 |
3 | Produces SVG files in `torchviz_exports/<backbone>/. Open them in any
4 | browser (SVG is vector - zoom freely).
5 |
6 | Difference from `export_onnx.py`:
7 | - ONNX/Netron shows the **architecture** (clean, module-level boxes).
8 | - torchviz shows the **autograd graph** (every add, matmul, layernorm,
9 |   softmax - the operations PyTorch will run during backward).
10 |
11 | The full-network graph is huge (thousands of ops). The script also renders
12 | focused sub-graphs (backbone, transformer, style encoder) which are far
13 | easier to read.
14 |
15 | For frozen backbones (DINOv2 default), the autograd graph stops at the
16 | backbone output - that's actually informative, since it shows exactly
17 | what's *trainable*. Pass `--unfreeze-backbone` to also see the inner
18 | backbone ops.
19 | """
20 |
21 | import argparse
22 | from pathlib import Path
23 |
24 | import torch
25 | from torchviz import make_dot
26 |
27 | from config import config as C
28 | from model.cvae import build_cvae

```

- The docstring contrasts ONNX/Netron architecture graphs with torchviz autograd graphs.
- `make_dot` traces operations connected to an output tensor, which can become very large.
- `config` and `build_cvae` supply the model and input shapes.

viz_torchviz.py:31-35 - render helper

```

31 | def render(out: Path, output_tensor: torch.Tensor, params: dict, label: str) -> None:
32 |     print(f" rendering {out.name}.svg ({label})")
33 |     dot = make_dot(output_tensor, params=params, show_attrs=False, show_saved=False)
34 |     dot.format = "svg"
35 |     dot.render(out.as_posix(), cleanup=True)

```

- `make_dot` creates a graph rooted at `output_tensor`.
- `params` labels trainable parameters in the graph.

- The graph is rendered as SVG and intermediate dot files are cleaned up.

viz_torchviz.py:38-55 - main setup

```

38 | def main(
39 |     out_dir: Path = Path("torchviz_exports"),
40 |     backbone: str = None,
41 |     freeze_backbone: bool = True,
42 | ) -> None:
43 |     cvae = build_cvae(backbone_name=backbone, freeze_backbone=freeze_backbone).eval()
44 |     backbone_name = cvae.backbone.backbone_name
45 |     out_dir = out_dir / backbone_name
46 |     out_dir.mkdir(parents=True, exist_ok=True)
47 |
48 |     print(f"built cvae: {sum(p.numel() for p in cvae.parameters()) / 1e6:.1f} M params "
49 |           f"(backbone={backbone_name})")
50 |
51 |     image = torch.zeros(1, C.NUM_CAMERAS, 3, C.IMAGE_HEIGHT, C.IMAGE_WIDTH, requires_grad=True)
52 |     qpos = torch.zeros(1, C.STATE_DIM, requires_grad=True)
53 |     actions = torch.zeros(1, C.CHUNK_SIZE, C.ACTION_DIM, requires_grad=True)
54 |     is_pad = torch.zeros(1, C.CHUNK_SIZE, dtype=torch.bool)
55 |

```

- The CVAE is built and output directory is organized by backbone name.
- Dummy image, qpos, and actions require gradients so torchviz has an autograd graph to follow.
- is_pad is boolean and does not require gradients.

viz_torchviz.py:58-78 - Focused backbone and style graphs

```

58 | # --- 1. Backbone only (smallest, most readable) ---
59 | feat, pos = cvae.backbone(image[:, 0]) # (B, 3, H, W)
60 | render(
61 |     out_dir / "01_backbone",
62 |     feat.sum(), # scalar so make_dot has a single root
63 |     dict(cvae.backbone.named_parameters()),
64 |     f"{backbone_name} + 1x1 projection + 2D sine pos embed",
65 | )
66 |
67 | # --- 2. Style encoder only (CVAE branch, qpos+actions -> mu/logvar) ---
68 | mu, logvar = cvae._encode_style(qpos, actions, is_pad)
69 | style_params = {}
70 | for name, p in cvae.named_parameters():
71 |     if any(k in name for k in ("style_", "cls_embed", "latent_proj")):
72 |         style_params[name] = p
73 | render(
74 |     out_dir / "02_style_encoder",
75 |     (mu.sum() + logvar.sum()),
76 |     style_params,
77 |     "CVAE style encoder: (qpos, actions) -> (mu, logvar)",
78 | )

```

- Backbone graph uses image[:, 0] because the backbone expects a standard image batch, not the model camera dimension.
- feat.sum() produces a scalar root, which simplifies autograd graph creation.
- The style graph directly calls _encode_style and filters parameters to those relevant to the style encoder.

viz_torchviz.py:80-103 - Full inference and training graphs

```

80 | # --- 3. Full inference forward (huge - every op in the deployment path) ---
81 | a_hat, _ = cvae(image, qpos, actions=None, is_pad=None)
82 | render(
83 |     out_dir / "03_inference_full",
84 |     a_hat.sum(),
85 |     dict(cvae.named_parameters()),
86 |     "Full inference path (backbone + main encoder + decoder + head)",
87 | )
88 |
89 | # --- 4. Full training forward (inference + style encoder branch) ---
90 | a_hat_t, (mu_t, logvar_t) = cvae(image, qpos, actions=actions, is_pad=is_pad)
91 | loss_proxy = a_hat_t.sum() + mu_t.sum() + logvar_t.sum()
92 | render(
93 |     out_dir / "04_training_full",
94 |     loss_proxy,
95 |     dict(cvae.named_parameters()),
96 |     "Full training path (inference + style encoder branch)",
97 | )
98 |
99 | print()
100 | print("done. open in any browser (SVG is vector):")
101 | for p in sorted(out_dir.glob("*.svg")):
102 |     size_kb = p.stat().st_size // 1024
103 |     print(f" {p} ({size_kb} KB)"

```

- The inference graph calls cvae without actions, showing deployment computation.
- The training graph calls cvae with actions/is_pad and uses a proxy scalar containing action, mu, and logvar outputs.
- The final loop prints generated SVG paths and sizes.

viz_torchviz.py:106-115 - CLI entry point

```
106 | if __name__ == "__main__":
107 |     p = argparse.ArgumentParser()
108 |     p.add_argument("--backbone", type=str, default=None,
109 |                   help="resnet18 | dinov2_vits14 | dinov2_vitb14 | dinov2_vitl14. "
110 |                       "Defaults to config.BACKBONE.")
111 |     p.add_argument("--unfreeze-backbone", action="store_true",
112 |                   help="DINOv2 backbones are frozen by default; pass this to render "
113 |                       "their internal autograd graph too.")
114 |     args = p.parse_args()
115 |     main(backbone=args.backbone, freeze_backbone=not args.unfreeze_backbone)
```

- The CLI mirrors export_onnx.py: choose a backbone and optionally unfreeze backbones to expose internal gradient paths.

Empty Package and Placeholder Files

Purpose. These files contain no executable code, but they still matter for package structure and future expansion.

- evaluate.py is currently empty. It is a placeholder for future offline evaluation or sanity-check scripts.
- config/__init__.py, data/__init__.py, and model/__init__.py are empty package markers. Python 3 can use namespace packages without them, but keeping them is conventional and explicit.
- Because these files have zero lines of logic, there is no line-by-line code to explain.

Generated Directories and Local Artifacts

Purpose. These directories are part of the working project but are not source code to study line-by-line.

- dataset/ is expected to contain logs/ with trial_N.csv files plus saved_frames/ and saved_videos/. It is the input data area.
- checkpoints/ stores policy_last.pt, policy_best.pt, numbered checkpoints, and dataset_stats.pkl generated during training.
- onnx_exports/ stores ONNX graphs produced by export_onnx.py for Netron viewing.
- torchviz_exports/ stores SVG autograd graphs produced by viz_torchviz.py.
- __pycache__/ folders are Python bytecode caches.
- .claude/settings.local.json is local tool permission metadata, not application logic.
- .codex is local Codex metadata and is ignored by git.

End-to-End Logic Summary

- Data starts as CSV rows and image files. data/dataset.py converts each timestep into normalized tensors and future action chunks.
- train.py creates splits and batches, then calls ACTPolicy with images, qpos, action chunks, and is_pad masks.
- ACTPolicy delegates prediction to ACTCVAE and converts predictions into masked L1 plus KL loss.
- ACTCVAE optionally encodes demonstration actions into a latent style during training; at inference it uses $z = 0$.
- Backbone turns microscope frames into image tokens. Transformer encodes latent/qpos/image tokens and decodes learned future-action queries.
- The action head returns a normalized action chunk. During rollout, ACTPolicy.inference denormalizes it back into robot target units.